



Full length article

Enhancing skin cancer detection through category representation and fusion of pre-trained models[☆]Lingping Kong^a, Juan D. Velasquez^{b,e}, Václav Snášel^{a,1,**}, Millie Pant^{c,f}, Jeng-Shyang Pan^{d,g}, Jana Nowakova^a^a Department of Computer Science, VSB-Technical University of Ostrava, listopadu 17, Czech Republic^b Department of Industrial Engineering, University of Chile, Santiago, O'Higgins 1058, Chile^c Department of Applied Mathematics and Scientific Computing, Indian Institute of Technology, Roorkee, Uttarakhand, 247667, India^d School of Artificial Intelligence, Nanjing University of Information Science and Technology, 219 Ningliu Road, China^e Instituto Sistemas Complejos de Ingeniería, Santiago, Chile^f Mehta Family School of Data Science and Artificial Intelligence, Indian Institute of Technology, Roorkee, Uttarakhand, 247667, India^g College of Computer Science and Engineering, Shandong University of Science and Technology, China

ARTICLE INFO

Keywords:

Pre-trained model
Sparse representation
Skin cancer data
Model interpretability

ABSTRACT

The use of pre-trained models in medical image classification has gained significant attention due to their ability to handle complex datasets and improve accuracy. However, challenges such as domain-specific customization, interpretability, and computational efficiency remain critical, especially in high-stakes applications such as skin cancer detection. In this paper, we introduce a novel interpretability-assisted fine-tuning framework that leverages category representation to enhance both model accuracy and transparency.

Using the widely known HAM10000 dataset, which includes seven imbalanced categories of skin cancer, we demonstrate that our method improves the classification accuracy by 2.6% compared to standard pre-trained models. In addition to precision, we also achieve significant improvements in interpretability, with our category representation framework providing more understandable insights into the model's decision-making process. Key metrics, such as precision and recall, show enhanced performance, particularly for underrepresented skin cancer types such as Melanocytic Nevi (F1 score of 0.94) and Actinic Keratosis (F1 score of 0.66).

Furthermore, the prediction accuracy of the proposed model of the top-3 reaches 98.21%, which is highly significant for medical decision making. This observation in interpretability underscores the value of top-*n* predictions, especially in challenging cases, to support more informed and accurate decisions. The proposed fusion framework not only enhances predictive accuracy, but also offers an interpretable model output that can assist clinicians in making informed decisions. This makes our approach particularly relevant in medical applications, where both accuracy and transparency are crucial. Our results highlight the potential of fusing pretrained models with category representation techniques to bridge the gap between performance and interpretability in AI-driven healthcare solutions.

1. Introduction

Skin cancer [1] is among the most common types of cancer, marked by abnormal and uncontrolled proliferation of skin cells [2]. It often results from exposure to ultraviolet radiation from the sun or tanning beds, leading to the formation of malignant tumors. Skin cancer remains a major cause of death worldwide. Dermatologists typically diagnose it through visual examinations, which have an accuracy rate

of around 60% [3]. The limitations of dermoscopy and the demand for higher diagnostic accuracy have fueled advances in computer-aided detection methods [4]. These systems generally follow five key steps: image acquisition, pre-processing, segmentation, feature extraction, and classification.

Medical image processing [5] involves the analysis and interpretation of medical images in 2D and 3D to aid diagnosis, treatment planning, and medical research [6]. Although 3D imaging techniques

[☆] The source code of the paper will be available at <https://github.com/lingping-fuzzy/cancer-model>.^{*} Correspondence to: o'higgins is Av Beauchef 851, office BP-06-19, Chile.^{**} Correspondence to: 17. listopadu 2172/15, 708 00 Ostrava - Poruba VSB-TUO, Czech Republic.E-mail addresses: lingping.kong@vsb.cz (L. Kong), jvelasqu@dii.uchile.cl (J.D. Velasquez), vaclav.snasel@vsb.cz (V. Snášel), pant.milli@as.iit.ac.in (M. Pant), jspan@cc.kuas.edu.tw (J.-S. Pan), jana.nowakova@vsb.cz (J. Nowakova).¹ Principal corresponding author

such as computed tomography (CT) and magnetic resonance imaging (MRI) are commonly used to visualize complex internal structures, 2D medical images, such as X-rays or dermoscopic images for the detection of skin cancer, are equally important. Medical image processing is crucial for the identification of diseases, the guide for surgical interventions, and the support of research by providing detailed information on anatomical and pathological conditions. It encompasses a variety of techniques, including image enhancement, segmentation, and analysis, tailored to improve precision in clinical decision-making.

The development of medical image processing has evolved significantly, especially with recent advances in artificial intelligence (AI), revolutionizing non-invasive diagnostics and making medical imaging an essential component of modern healthcare [7]. AI has enabled more accurate and automated image analysis, improving disease detection, treatment planning, and clinical decision making. However, medical images pose unique challenges compared to other types of images due to their complexity, variability, and the critical need for precision [8]. Medical images often contain subtle features, such as small tumors or abnormalities, that require specialized algorithms for accurate detection to extract the particular ‘subtle’ feature. Furthermore, factors like varying image quality and differences in imaging modalities make it challenging to process medical images with the same techniques used for general images. Addressing these challenges requires AI-driven solutions that can handle the nuances of medical images.

There are some very popular and well-known pre-trained models designed for medical data.

- CheXNet [9] (<https://github.com/arnoweng/CheXNet>): A model proposed by Stanford ML Group, trained on chest X-rays images to detect pneumonia, backbone with a 121-layer convolutional neural network.
- DeepMind’s RetinaNet [10,11] (<https://github.com/facebookresearch/Detectron?tab=readme-ov-file>): A model trained on retinal images to detect diabetic retinopathy, its architecture is based on ResNet CNN backbone.
- VNet [12] (<https://github.com/mattmacy/vnet.pytorch>): A volumetric (3D) extension of the UNet for 3D medical image segmentation, where UNet (<https://lmb.informatik.uni-freiburg.de/people/ronneber/u-net/>) is a convolutional network architecture for biomedical image segmentation.
- Med3D [13] (<https://github.com/Tencent/MedicalNet>): A collection of 3D neural network models pre-trained on large-scale medical datasets.
- DenseNet121 [14] (<https://pytorch.org/vision/stable/models>) pre-trained in CheXpert (<https://stanfordmlgroup.github.io/computations/chexpert/>): A DenseNet model trained on the CheXpert dataset for chest radiograph analysis.
- BioBERT [15] (<https://github.com/dmis-lab/biobert>): primarily a language model focusing on biomedical text mining that includes entity recognition, relation extraction, question answering, etc., it has been used for various biomedical text tasks, including radiology report analysis.
- 3D ResNet [16] (https://pytorch.org/hub/facebookresearch_pytorchvideo_resnet/): A 3D version of ResNet, used for volumetric medical data.
- nnU-Net [17] (<https://github.com/MIC-DKFZ/batchgenerators>): A self-adapting framework for biomedical image segmentation that automatically configures itself for different datasets.

In clinical applications, neural networks have significantly facilitated the classification of disease categories, particularly with the advent of pre-trained models, ensemble models, and foundation models. These models are especially useful in situations that involve small datasets, enabling them to achieve reasonable results in clinical databases. However, several challenges remain in the skin cancer detection issues.

The classification of medical image data faces several critical challenges that require innovative solutions to ensure efficient and effective results. First, many state-of-the-art models used in medical image analysis have complex architectures, making re-training and fine-tuning computationally expensive and storage-intensive. The high number of epochs needed to achieve accurate results significantly increases both computational power and memory requirements. Reducing these costs is essential for broader generalization and practical applications. Second, medical images often contain subtle features that are more difficult to identify compared to general image classification tasks. These features require specialized techniques for detection and classification. Furthermore, medical data sets are frequently imbalanced, with insufficient amounts of data for certain conditions, making it difficult to train models that perform well on minority classes. Addressing this imbalance and developing strategies to improve performance with limited and underrepresented medical data is a significant challenge.

Lastly, the interpretability of the model is a crucial factor in medical imaging. Beyond producing accurate predictions, models must provide information that helps healthcare professionals understand the reasoning behind the results. This interpretability helps physicians make informed decisions about further diagnosis and treatment. Managing these challenges can lead to more efficient and accessible medical image processing solutions, ultimately improving patient care and medical outcomes.

Summary of Motivation

Enhancing pre-trained models is of paramount importance due to their limitations in domain-specific tasks. Although these models are efficient and easy to implement, they often face difficulties with specialized datasets, show potential biases, and lack interpretability, particularly in fields such as medical imaging where understanding the reasoning behind predictions is crucial for diagnosis and treatment.

There are several challenges in applying existing interpretability algorithms to medical datasets:

1. **Gradient-based interpretability methods**, such as SHAP <https://shap.readthedocs.io/en/latest/> and Captum https://captum.ai/docs/attribution_algorithms, use input gradients to provide attribution or highlight important nonlinear activations. However, in skin cancer data, crucial features often contribute equally, resulting in entire images being highlighted with minimal differentiation. This makes it difficult to apply these highlighted regions for accurate cancer identification.
2. **Certain interpretability approaches**, such as Lime and GradientSHAP, are designed for tabular data, where they assign importance to individual features. However, when applied to image data, these methods face high computational costs due to the need for large candidate datasets, often leading to memory allocation errors. Additionally, small sample support and layer analysis visualization can result in overly complex, all-relevant maps that create biased and inefficient outcomes in medical data.
3. **A fundamental limitation of existing classification-based interpretability methods** is the lack of a comprehensive analysis. Crucial information, such as the confidence level of the inference, the likelihood of potential errors, and the recommendations of the top-*n* results, is often absent. Visualizing and interpreting these metrics would be much more useful for medical professionals than simply providing raw numerical output.

Fusion techniques help overcome these challenges by combining well-pre-trained models with interpretable category representation methods. This improves both model performance and interpretability, making predictions more accurate and transparent, ultimately improving decision making and patient care. More importantly, we optimize computational and resource efficiency by incorporating category representation into the models, enhancing both their generalization and applicability across various tasks.

Summary of Contributions

In this paper, we focus on the study of skin cancer data, the most common cancer and one of the leading causes of death worldwide. The standard HAM10000 dataset includes seven imbalanced categories of skin cancer. To address the aforementioned challenges, we propose an interpretability-assisted fine-tuning technique to enhance the performance of baseline models. Specifically, we apply category representation derived from sparse representation to identify non-active neurons in pre-trained model, which guide the fine-tune process in low-cost and improve the performance in specific task. The category representation encapsulates essential information from each type of skin cancer, and the computational complexity remains low. The re-training process focuses on nonactive neurons rather than the entire network architecture, making the process more efficient and concise.

- We propose an interpretability-assisted fine-tuning framework to enhance the performance of pre-trained models for specific tasks.
- We introduce a category representation method that constructs a Library to represent skin cancer data categories and interpret the structural relationships within the data. Instead of solving complex matrix equations in sparse (category) representation, we propose a fast aggregation technique based on nearest neighbors to reduce computation costs.
- The category representation is built from the features of the learned data set, which provide insights into model performance and help identify inactive neurons, helping improve the model.
- To efficiently extract relevant features and identify inactive neurons in pre-trained models from high-dimensional data, we propose the application of random projection using a custom projection matrix and hard principal component thresholding, significantly reducing storage and computational costs.

The remainder of this paper is structured as follows. Section 2 provides an overview of basic medical image processing techniques and reviews commonly used pretrained models in the medical domain, along with an introduction to sparse representation and classification evaluation metrics. Section 3 presents the proposed interpretability-assisted model update framework. In Section 4, we evaluate the framework using two pre-trained models with skin cancer data, and compare the interpretability and model's improvement with the WeightWatcher tool. Finally, conclusions are presented in Section 5.

2. Preliminary

This section introduces image processing techniques, highlights the pre-trained models, and then reviews the compressed sensing and analysis of the principal components, which we will adopt and adapt to our proposed framework.

2.1. Medical image processing techniques

This section we will introduce commonly used medical image processing techniques and state-of-the-art models, as this paper addresses the 2D skin cancer data, we will highlight the techniques applied to 2D images on a skin cancer dataset.

The above introduced CheXNet to nnU-Net, most common used model representatives are mainly built on a convolutional network, which is well-suited to capture spatial hierarchies and patterns in images through local connectivity and parameter sharing, meanwhile captures learn translation-invariant features, recognizing objects regardless of their position. The deep learning model is significantly affected on its performance by volume of high-quality dataset. Building enough high-quality dataset is highly challenging in medical imaging because data acquisition and annotation are both difficult tasks. Hence, there are many researchers they apply pre-trained model with convolutional network backbone to learn hierarchical representations —

from edges to complex shapes — makes them highly effective for image classification tasks.

Table 1 presents a review of deep neural network models applied to the HAM10000 skin cancer dataset, which consists of seven classes. These methods primarily employ a feature-level approach to enhance the representation of each skin cancer type. Additionally, some studies integrate annotation-based segmentation techniques [30,31] for region-of-interest analysis. The complementary nature of these approaches has the potential to improve overall performance.

2.2. Evaluation and metrics

We assess the model's performance using several essential metrics, such as *Accuracy*, which reflects the overall accuracy of the model's predictions, and *Precision*, which assesses the proportion of true positive predictions among all positive predictions. *Recall* (or sensitivity) is used to gauge the ability of the model to identify all relevant instances within a dataset. These metrics are summarized in a *classification report*, which provides a comprehensive breakdown of performance for each class, including the *F1-score* serves as the harmonic mean of precision and recall, effectively balancing the relationship between false positives and false negatives.

In addition to numerical evaluation, we also assess the model performance using *visualization techniques* such as cluster embedding analysis. By visualizing *cluster similarity*, we can examine how well the model groups similar classes together, helping to verify whether the learned feature space effectively captures the relationships between different skin lesion types. For example, we may use UMAP to project high-dimensional data onto a 2D plane, allowing us to observe how distinct or overlap the clusters of different lesion categories are. These visual assessments complement the numerical metrics and offer deeper insight into the model's ability to generalize and differentiate between complex patterns in the dataset, as well as the visualization of the category representation for the library.

The localization of the injuries are identified by 15 regions such as 'scalp', 'face', 'ear', 'neck', 'abdomen', 'chest', 'upper extremity', 'unknown', 'back', 'trunk', 'foot', 'lower extremity', 'genital', 'hand', and 'acral'. Fig. 1 presents a pie chart of the types of lesion by localization. In particular, in the 'acral' region, only the *melanocytic nevi* type of skin cancer was recorded. Additionally, *Melanocytic nevi* is the most common skin cancer type found at all locations in the data set. For visualization, we filter out it from other localizations to observe the other types of recording. For ear localization, aside from *Melanocytic nevi*, melanoma is the most prevalent, accounting for 65.4% of cases, followed by benign keratosis and then actinic keratoses. In this data set, no other types of skin cancer are recorded for the ear localization. These metadata suggest a potential relationship between skin cancer types and their localizations, which could be considered in future classification efforts, which would be illustrated in Section 4.

2.3. Sparse approximation

Sparse representation involves expressing a signal as a linear combination of a few fundamental components, or "atoms", from an over-complete dictionary. This concept has diverse applications, including in image processing, machine learning, and medical imaging. The solution of the Sparse representation in the application involves compressed sensing. Meanwhile, sparse representation is also the key Compressed sensing, which is a method that efficiently acquires and reconstructs signals by solving underdetermined linear systems, assuming signals are sparse or can be sparsely represented in a particular domain. By exploiting this sparsity, compressed sensing recovers signals from fewer measurements than typically required.

The steps to compute a sparse representation include:

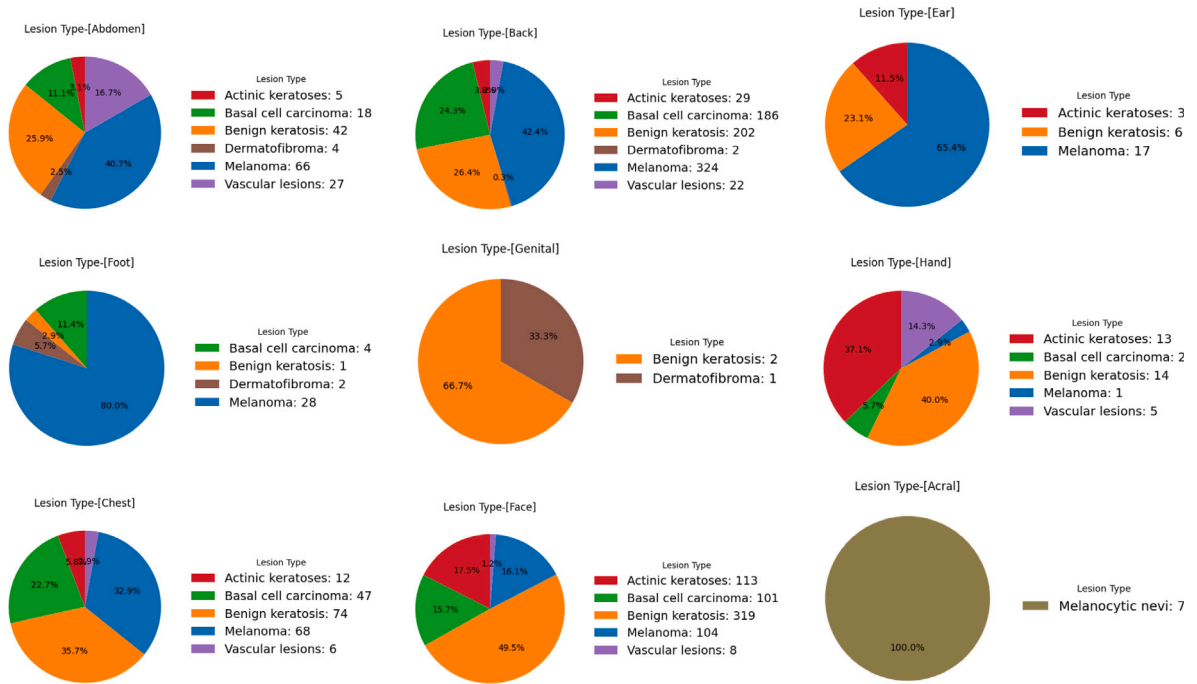
- Define the Dictionary: Construct an over-complete dictionary of atoms that exceeds the signal's dimensionality.

Table 1

Literature about deep neural network models addressing skin cancer dataset HAM10000 with seven classes (see [18,19]).

Paper	Base model	Preprocess	Data size	Fusion tech
Ali 2022 [20]	EfficientNets B0–B7	Hairs removal, augmentation	Total-10015 (72-8-20)	Dense batch normalization, dropout layers
pant 2022 [21]	ResNet50	Translational, specular and rotational techniques	Balanced 1500 × 7	Xception
Shahin 2021 [22]	Deep CNN	Filter or kernel for remove noise and artifacts; normalize input	Two classes (85-15)	Feature Extraction -
Duyen 2021 [23]	ResNet50	Augmentation (rotating, flipping, cut out, cropping), Stratified sampling	Total-10015 (68-17-15)	VGG16 and MobileNet with class-weighted and focal loss
Dibaloke 2023 [24]	Deep CNN	Augmentation (rotation, shift, flip, shear, rescale, fill) Pixel standardization	Total-50024 Augmented	ENSEmble deep network
Gomathi 2023 [25]	U-Net	Adaptive median filter	Total-10015 (80-20)	Bacterial Foraging and Particle Swarm Optimization
Essam 2024 [18]	Deep CNN	Normalization and image reshaping.	Total-10015 (80-20)	Random Oversampling
Carolin 2024 [26]	Vision Transformer	exploratory data analysis	Total-9948 (80-10-10)	–
Omneya 2024 [27]	Inception	Data augmentation, PCA, feature selection, LIME	Total-10015 (70-30)	Xception, ResNet-50, and ResNet-101
Alenezi 2023 [19]	Resnet101	Haar wavelet transformation pooling, and normalization	Total-10015 (80-20)	Extreme Learning Machine.
DMSFF 2024 [28]	ResNet50	Augmentation (flip, rotation, normalize)	(10015+) (70-30)	Diffusion model generates new samples (+ synthetic data)
Ling 2024 [29]	ResNet50	Augmentation (translation and rotation)	Total 15000 (NA)	Attention mechanism
Proposed	ResNet18/ EfficientNet	Augmentation (flip, rotation, normalize)	Total-7470 (70-15-15)	Category representation aiding interpretability

CNN: convolutional neural network; LIME: local interpretable model.

**Fig. 1.** Pie chart of Lesion type for localization; Lesion Type-[Localization], including, ear, back, face, foot, etc. Note: we filter out the *Melanocytic nevi* type counts, as it appears in all candidate body area.

- **Formulate the Optimization Problem:** Set up an optimization problem to represent the signal as a sparse combination of dictionary elements, minimizing nonzero coefficients while maintaining accuracy.
- **Solve the Optimization Problem:** Using algorithms such as Matching Pursuit (MP), Orthogonal Matching Pursuit (OMP), Basis Pursuit (BP), or LASSO to iteratively select atoms that best approximate the signal.

- **Reconstruction:** Reconstruct the signal by combining the selected atoms with their corresponding coefficients.

This process allows compressed sensing to efficiently recover signals by leveraging their sparse structure. However, to accelerate the computation for identifying cluster or class membership, we adopt sparse representation but forego solving the optimization problem. Instead, we identify the majority label of the nearest neighbors as candidates, which will be explained in detail in Section 3

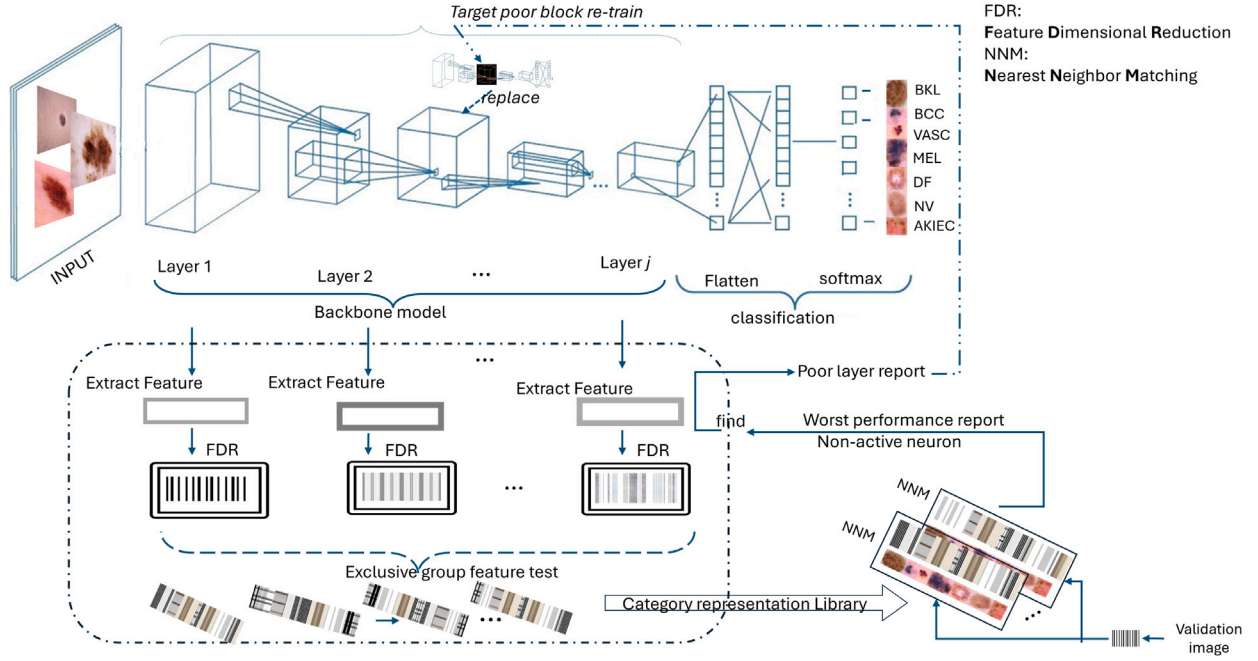


Fig. 2. The flowchart of the proposed framework.

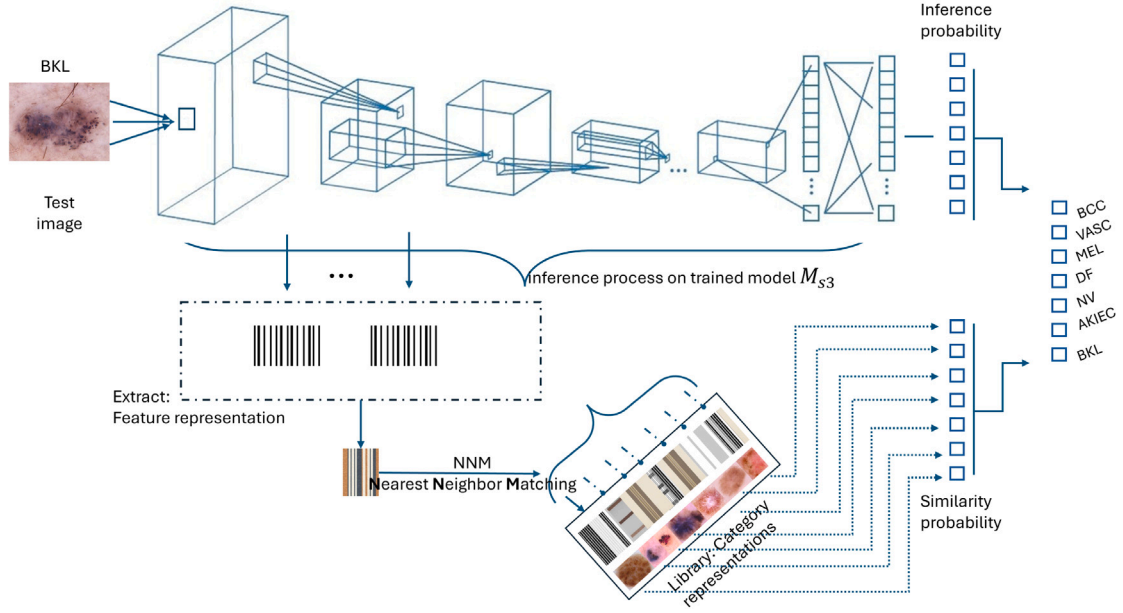


Fig. 3. The procedure of skin cancer data inference by the model and interpretable category representation-library. Abbr refers to Fig. 2.

In our work, we tackle these issues by developing image processing algorithms extracting representations and integrating interpretability into AI models. This helps ensure reliable and transparent medical diagnoses, ultimately improving patient care quality (see Fig. 3).

3. Interpretability-assisted model-update framework

This section presents the proposed interpretability-assisted model update framework in three steps: (1) the overall architecture with visual flow, (2) Category representation for interpretability, including neurons capture and representation construction, (3) Model Update and Inference, and then (4) Efficient Neuron Extraction via Random Projection. Steps 2 and 3 constitute the primary components of the

framework, while step 4 is introduced to manage high-dimensional intermediate feature spaces.

In summary, Step 2 uses a register hook technique to capture neurons from the model, allowing us to extract intermediate latent features at specific layers for given inputs. Meanwhile, this step constructs category representations by stacking active neurons (transformed features) extracted from inputs belonging to the same class. This representation allows us to assess the class of test data. For cases where the model generates high-dimensional intermediate features, which increases computational and storage burdens, Step 3 introduces an efficient neuron extraction method using random projection for dimensionality reduction. Step 4 instructs us how to perform inference for test data (see Fig. Fig. 3).

The following subsections provide a detailed explanation of each step.

3.1. Overall architecture

The overall architecture of the proposed framework is illustrated in Fig. 2. To simplify the discussion, we consider an n -layer model with an input image batch denoted as X^0 . The output of the first layer is X_{s-}^1 , and the output of the final layer is X_{s-}^n . The $(i+1)$ th layer operates on the output of the previous i th layer, as expressed by

$$X_{s-}^{i+1} = \text{Layer}^{i+1}(X_{s-}^i),$$

where each layer represents operations in the backbone model (e.g., pooling, activation layers). Note that the data shape (or data dimensions) of X_{s-}^{i+1} and X_{s-}^i may differ due to various layer operation. Let $\ell = \{1, 2, \dots, c\}$ represent the set of class labels, where c is the number of classes.

Some key concepts help in understanding the framework (see Table 2).

- The subscript $s-$ distinguishes the stage of the model. For instance, the pre-trained backbone model in stage one is denoted as M_{s1} , and its first layer output as X_{s1}^1 . After fine-tuning, the model is updated and denoted as M_{s2} , etc., the model will be renamed after each update by its subscripts.
- Category representations, or the *library*, are constructed using the training dataset, and active neuron verification is performed on the validation data.
- Fine-tuning occurs twice. Initially, the entire pre-trained model undergoes fine-tuning. The second fine-tuning focuses on non-active neurons, as determined by the category representation process, both using few epochs to minimize computational cost.

The process starts with input images (e.g., skin cancer images) resized to fit the backbone model requirements. The backbone model, which is pre-trained (e.g. ResNet18 or EfficientNet in our experiments), consists of several layers (layer-1, layer-2, ..., layer- n , depending on the chosen model). The default linear classification layer of the backbone model is replaced with a fully connected layer, customized for the dataset's number of classes (e.g., seven for the skin cancer dataset). This initial model M_{s1} is quickly fine-tuned to obtain M_{s2} .

We then extract intermediate feature representations $\{X_{s2}^i, \dots, X_{s2}^j\}$ from M_{s2} using the register hook method, where the user selects the feature layers i and j . Active neurons from these layers are grouped according to their true and correctly predicted class labels, forming the *category representation* as shown in Fig. 2. The next step is to verify these representations through validation, constructing a stack of candidate category groups.

To identify the worst-performing category group, we evaluated the validation set using sparse representations. The category representations, formed from the training data, serve as a *library*, while the validation image acts as a *query*. Sparse coefficient vectors ideally have large values in the regions of the library corresponding to the correct class. The worst category group indicates the region of non-active neurons, leading to a second fine-tuning step where only the non-active neuron regions are updated, resulting in the final model M_{s3} .

For inference, we combine the decision probabilities from both the backbone model and category representation using a weighted average. The highest probability determines the prediction label. This process enhances the baseline model's performance on custom datasets by leveraging category interpretability and targeting non-active neurons for fine-tuning.

3.2. Category representation for interpretability (CPI)

Category representation simulates sparse representations, derived from compressed sensing. It constructs a label-based *library* from the

feature representations $\{\hat{X}_{s2}^i, \dots, \hat{X}_{s2}^j\}$ as follows:

$$\hat{X}_{s2}^i = \text{flatten}(X_{s2}^i)_{\text{batch_size} \times m}$$

Here, *flatten* collapses the data into a one-dimensional representation. For an input batch X^0 of size b , we obtain b one-dimensional representations.

All one-dimensional data from the training set is stacked and grouped by class label to form the *library* $\Phi = \{\tilde{X}_{s2}^1, \tilde{X}_{s2}^2, \dots, \tilde{X}_{s2}^c\}$, where subscripts 1 to c represent class labels in ℓ . For inference on a validation image y , we first compute its one-dimensional representation and then perform a g -nearest neighbors search in Φ . The majority class label of the neighbors is the predicted class of y :

$$\phi_{ind} = \text{nearest_neighbor}(\text{flatten}(y_{s2}^i), \Phi)$$

$$k \leftarrow \text{find}(\phi_{ind}, \ell)$$

The class $k \in \ell$ is assigned based on the maximum occurrence of indices in ϕ_{ind} .

3.3. Model update and inference

The category representation process identifies non-active neuron regions, corresponding to specific layers. A second fine-tuning step updates only the non-active layers, resulting in the model M_{s3} .

This category representation block performs inference for new data, as presented in previous step. Additionally, the inference result can be expressed as a weight vector showing the probability of potential category $\mathbf{p}^\ell = \{p_1^\ell, \dots, p_c^\ell\}$, denoted as

$$p_k^\ell = \frac{\text{size}\{\phi_{ind} == k\}}{|\phi_{ind}|}$$

where $k \in \{1, 2, \dots, c\}$ and $|\phi_{ind}|$ is the number of neighbors. The final prediction is obtained by combining the results from M_{s3} and the category representation, weighted by the accuracy of the validation.

During inference, the output probability of each class for one test data is computed as:

$$k \leftarrow \text{softmax}(\text{val}^\ell \mathbf{p}^\ell + \text{val}^m M_{s3})$$

where, M_{s3} output the final probability set for each label, and val^ℓ and val^m are the validation accuracy by the category representation inference and the M_{s3} model, respectively.

3.4. Efficient neuron extraction via random projection

For high-dimensional feature spaces, we propose a random projection method for efficient neuron extraction. This method uses a projection matrix and principal components based on hard thresholding, ensuring precision and computational efficiency.

In the process of CPI, we apply the flatten function to get a one-dimensional representation for each image data. However, The size or shape of X_{s2}^i depends on the i th layer operation, which may lead to this m a large number and result in a large size of *library*. To avoid the large-data computation, we perform a random projection that maps the large-dimensional data to low-dimensional data, formulated as below:

$$\{\hat{X}_{s2}^i\}_{b \times p} = \hat{X}_{s2}^i \times R_{m \times p}$$

where, the extracted $\hat{X}_{s2}^i$ representation data has a shape of $\text{batch_size} \times m$, $R_{m \times p}$ is a random matrix with 0 or 1 values in the entry, then the result of two matrix multiplication ($\times$) is a size of $\text{batch_size} \times p$, that we set $p \ll m$.

Note: we generate the random matrix R by initializing a zero matrix of size $m \times p$, then random selects r percent of elements and switches the zero entry to one.

Table 2

Annotation of symbols used throughout the paper.

Symbols	Annotation
b or batch_size	batch_size parameter assign for training model.
i, j and k	i, j are used for layer indices, and mostly k is used for label index.
ℓ	It represent the data classification label set $\ell = \{1, 2, \dots, c\}$, or symbol for label related index.
m, p	feature dimension, $p \ll m$
X_{s-}^i	Obtained initial feature data on X in layer i of model M_{s-}
$\hat{X}_{s-}^i$	flatten feature data on X_{s-}^i
$\hat{X}_{s-}^{\text{label}-k}$	The construct element of <i>library</i> , the category representation for one label k from model M_{s-}
$R_{m \times p}$	A random projection matrix size of $m \times p$
M, M_{s-}	Neural network model, the baseline model or (pre-trained) backbone model M_{s1} , retrained model M_{s2} .

4. Experiments

4.1. Dataset

The HAM10000 (“Human Against Machine with 10,000 training images”)² dataset contains 10,015 dermoscopic images used to detect pigmented skin lesions. This publicly available dataset is hosted by the International Skin Imaging Collaboration (ISIC) archive [32] and is widely used for academic machine learning research. It includes images from key diagnostic categories:

- 6705 images of Melanoma (Mel)
- 1113 images of Melanocytic Nevus (NV)
- 514 images of Basal Cell Carcinoma (BCC)
- 1099 images of Benign Keratoses (solar lentigo, seborrheic keratosis, and lichen planus-like keratosis) (BKL)
- 327 images of actinic keratosis and intraepithelial carcinoma (Bowen’s disease) (AKIEC)
- 142 images of Vascular lesions (angiomas, angiokeratomas, pyogenic granulomas, and hemorrhages) (VASC)
- 115 images of Dermatofibroma (DF)

The dataset offers a comprehensive collection of common pigmented lesions, making it a valuable resource for training machine learning models in dermatological diagnostics.

4.2. Experiment setting

For a fair comparison and reproducibility, we maintain consistent experimental settings across all models, including the baseline (backbone, Resnet18 and EfficientNet) and fine-tuned models. Specifically, we use ResNet18 with ResNet18_Weights.IMAGENET1K_V1 from torchvision.models, and EfficientNet_b0 with default weights from torchvision.transformers. The detailed hyperparameters and settings are as follows.

To align with the input requirements of baseline models, images are resized to 256×256 pixels, with data augmentation techniques including RandomHorizontalFlip, RandomRotation, and Normalize (mean = [0.485, 0.456, 0.406], std = [0.229, 0.224, 0.225]). For fine-tuning, we set 10 epochs, a learning rate of 0.0001, and a batch size of 32. To mitigate class imbalance, we employ a WeightedRandomSampler with replacement in the dataloader. The Adam optimizer is used for model training.

To accommodate the seven-class classification task, we modify the classifier of the baseline model by adding two fully connected layers, the first layer having an input size of 64 and the output layer providing probability scores for the seven classes.

For data splitting, after removing duplicates, we retain 7470 images. Of these, 70% (5229 images) are used for training, 15% (1120 images) for validation, and the remaining images for testing.

4.3. Metric evaluation results

This section presents the metric evaluation for models and corresponding results.

4.3.1. Description of model and library

M_{s1} represents the baseline (pre-trained) model, in which we replace the classifier layer with two fully connected layers, outputting a seven-class probability score. M_{s2} refers to the thoroughly fine-tuned model over ten epochs. Afterward, we construct the Library (category representation) using inference feature data from M_{s2} to identify underperforming neurons. A targeted fine-tuning process is applied to these neurons over ten epochs to create M_{s3} . To identify poorly performing layers, inference features from specific layers are extracted and matched to the corresponding constructed Library, denoted as Library- i .

1. M_{s1} , M_{s2} , and M_{s3} refer to the pre-trained model, the fully fine-tuned model, and the targeted re-tuned model, respectively.
2. Library-1, Library-2, and Library- i represent the stacked category representations of inference features from specific layers in the model.
3. To identify underperforming layers, we extract inference features and build the library using M_{s2} on the training data. To support the final classification, inference features are extracted, and the Library is constructed using M_{s3} on the training data.

In addition, we ignore the evaluation of M_{s1} due to its random weights in the last fully-connected layer, leading to poor random results.

4.3.2. Classification results of ResNet

We construct layer-specific Libraries from model M_{s2}^{ResNet} , which is derived from the pre-trained ResNet18. First, we extract inference features from layer-1, layer-2, and layer-3 using the training dataset, and then group the selected features to build category representation, further to construct Library-1, Library-2, and Library-3. We then evaluate the classification performance of each Library on the validation data, as shown in Table 3. The classification results are 0.55, 0.533, and 0.530 for Library-1, Library-2, and Library-3, respectively, indicating that the third layer is underperforming. Therefore, we fine-tune M_{s2}^{ResNet} on the third layer and fixed all other layers to create the M_{s3}^{ResNet} model. The confusion matrices of classification result for M_{s2}^{ResNet} and M_{s3}^{ResNet} on the test data are presented below.

$$C_{ResNet}^{M_{s2}} = \begin{matrix} & \begin{matrix} AKIEC & BCC & bkl & DF & NV & MEL & VASC \end{matrix} \\ \begin{matrix} AKIEC \\ BCC \\ bkl \\ DF \\ NV \\ MEL \\ VASC \end{matrix} & \begin{pmatrix} 18 & 4 & 6 & 0 & 4 & 2 & 0 \\ 3 & 37 & 3 & 0 & 5 & 0 & 1 \\ 5 & 2 & 75 & 0 & 18 & 9 & 0 \\ 0 & 1 & 1 & 4 & 5 & 0 & 0 \\ 1 & 1 & 13 & 0 & 779 & 17 & 0 \\ 1 & 2 & 10 & 0 & 40 & 38 & 1 \\ 0 & 2 & 0 & 0 & 2 & 0 & 11 \end{pmatrix} \end{matrix}$$

² Data download link <https://dataverse.harvard.edu/dataset.xhtml?persistentId=doi:10.7910/DVN/DBW86T>.

Table 3Category representation (Library) experiment on validation data for targeting the poor performance layer in fine-tuning Resnet18 M_{s2} .

Classification report for Layer-1 by constructed Library-1					Classification report for Layer-2 by constructed Library-2				
Type	Precision	Recall	F1-Score	Support	Type	Precision	Recall	F1-Score	Support
AKIEC	0.19	0.32	0.24	34	AKIEC	0.16	0.24	0.19	34
Basal(BCC)	0.13	0.22	0.16	49	Basal(BCC)	0.10	0.22	0.14	49
bkl	0.20	0.43	0.28	109	bkl	0.23	0.45	0.31	109
DF	0.00	0.00	0.00	11	DF	0.08	0.18	0.11	11
NV	0.87	0.66	0.75	810	NV	0.89	0.62	0.73	810
Melanoma	0.20	0.21	0.21	92	Melanoma	0.22	0.27	0.25	92
Vascular	0.03	0.07	0.04	15	Vascular	0.02	0.07	0.03	15
Macro avg	0.23	0.27	0.24	1120	Macro avg	0.24	0.29	0.25	1120
Weighted avg	0.68	0.55	0.60	1120	Weighted avg	0.70	0.53	0.59	1120
Accuracy			0.55	0	Accuracy			0.533	0
Classification report for Layer-3 by constructed Library-3									
Type	Precision	Recall	F1-Score	Support					
AKIEC	0.15	0.24	0.18	34					
Basal(BCC)	0.08	0.18	0.12	49					
bkl	0.18	0.36	0.24	109					
DF	0.08	0.09	0.09	11					
NV	0.89	0.64	0.74	810					
Melanoma	0.17	0.18	0.18	92					
Vascular	0.08	0.27	0.13	15					
Macro avg	0.23	0.28	0.24	1120					
Weighted avg	0.69	0.53	0.59	1120					
Accuracy			0.530	0					

$$C_{ResNet}^{M_{s3}} = \begin{matrix} & AKIEC & BCC & bkl & DF & NV & MEL & VASC \\ \begin{matrix} AKIEC \\ BCC \\ bkl \\ DF \\ NV \\ MEL \\ VASC \end{matrix} & \begin{pmatrix} 28 & 2 & 2 & 0 & 1 & 1 & 0 \\ 6 & 38 & 1 & 0 & 4 & 0 & 0 \\ 8 & 1 & 75 & 0 & 19 & 6 & 0 \\ 1 & 0 & 1 & 5 & 4 & 0 & 0 \\ 5 & 6 & 7 & 0 & 773 & 19 & 1 \\ 3 & 1 & 9 & 0 & 35 & 42 & 2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 15 \end{pmatrix} \end{matrix}$$

M_{s2}^{ResNet} correctly classified 779 cases of the NV type of skin cancer, six more than M_{s3}^{ResNet} , which correctly classified 773 cases. Both models correctly classified 75 cases of the BKL type. However, for the remaining skin cancer types, M_{s3}^{ResNet} outperformed M_{s2}^{ResNet} by successfully classifying more cases, particularly improving the precision for skin cancer types labeled minority. As shown in Table 4, M_{s2}^{ResNet} achieved an accuracy of 0.85, while M_{s3}^{ResNet} reached 0.87. In terms of the recall metric, M_{s3}^{ResNet} demonstrated improvements across most skin cancer classifications. Furthermore, when we fused M_{s3}^{ResNet} with Library-1 (which achieved the highest score compared to Library-2 and Library-3), the fused model achieved a classification accuracy of 0.8724. Although this represents only a slight improvement over M_{s3}^{ResNet} , the fusion process also improved the model's interpretability, which will be discussed in further detail in the next subsection. The results presented above are based on one of ten test runs, with the following results: 87.24, 86.17, 86.26, 85.99, 87.15, 86.98, 86.80, 86.62, 86.44, and 87.06, resulting in an average accuracy of 86.67 and a standard deviation of 0.419, then we infer the confidence interval ($CI = \mu + Z \frac{\sigma}{\sqrt{n}}$) of the proposed model is $86.67 \pm 1.96 \times 0.419/\sqrt{10} = 86.67 \pm 0.259$. The pretrained model, along with the fine-tuned model aimed at nonactive neurons, is available at <https://github.com/lingping-fuzzy/cancer-model>. Researchers can use the provided code to evaluate performance and analyze layer weight differences.

4.3.3. Classification results of EfficientNet

We constructed layer-specific Libraries from model M_{s2}^{EffNet} following the same process used for ResNet. The Library-specific classification reports are shown in Table 5. The three Libraries — Library-1, Library-2, and Library-3 — produced classification accuracies of 0.65, 0.72, and 0.79, respectively, indicating that layer-1 performed the worst, while

layer-3 performed the best. Consequently, we fine-tuned M_{s2}^{EffNet} on layer-1 while keeping all other layers fixed for ten epochs, resulting in model M_{s3}^{EffNet} .

The classification confusion matrices for M_{s2}^{EffNet} and M_{s3}^{EffNet} are presented below, where M_{s3}^{EffNet} shows improved performance, particularly for minority-labeled skin cancer types, including AKIEC, BCC, and DF. Additionally, it significantly enhances the classification of the NV type. Overall, M_{s3}^{EffNet} achieved a 2.2% improvement in accuracy compared to M_{s2}^{EffNet} . Furthermore, we applied the fused method, combining M_{s3}^{EffNet} with the Library, and the results are shown in Table 6. In total, M_{s3}^{EffNet} outperformed M_{s2}^{EffNet} , and the fusion model provided an additional improvement of around 0.3%.

In conclusion, the fusion of category representation enhances the baseline model's classification performance with minimal resource cost, and we will further demonstrate how the Library aids in the interpretability of results.

$$C_{EfficientNet}^{M_{s2}} = \begin{matrix} & AKIEC & BCC & bkl & DF & NV & MEL & VASC \\ \begin{matrix} AKIEC \\ BCC \\ bkl \\ DF \\ NV \\ MEL \\ VASC \end{matrix} & \begin{pmatrix} 22 & 0 & 6 & 0 & 1 & 5 & 0 \\ 3 & 42 & 1 & 0 & 2 & 0 & 1 \\ 3 & 1 & 80 & 0 & 11 & 14 & 0 \\ 1 & 2 & 2 & 3 & 2 & 1 & 0 \\ 3 & 2 & 25 & 0 & 734 & 46 & 1 \\ 4 & 4 & 7 & 0 & 16 & 61 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 15 \end{pmatrix} \end{matrix}$$

$$C_{EfficientNet}^{M_{s3}} = \begin{matrix} & AKIEC & BCC & bkl & DF & NV & MEL & VASC \\ \begin{matrix} AKIEC \\ BCC \\ bkl \\ DF \\ NV \\ MEL \\ VASC \end{matrix} & \begin{pmatrix} 24 & 0 & 5 & 0 & 2 & 2 & 1 \\ 2 & 43 & 1 & 0 & 2 & 0 & 1 \\ 5 & 3 & 77 & 2 & 14 & 8 & 0 \\ 2 & 1 & 1 & 5 & 1 & 0 & 1 \\ 3 & 2 & 18 & 0 & 757 & 27 & 4 \\ 3 & 2 & 8 & 2 & 19 & 57 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 15 \end{pmatrix} \end{matrix}$$

4.4. Layer quality watcher

Our proposed category representation framework aids in identifying underperforming layers in a trained model by analyzing the feature space produced from both the training and validation data. This allows for a targeted improvement of specific layers that do not contribute effectively to the model's overall performance.

Table 4

The classification report on testing data by M_{s2}^{ResNet} , M_{s3}^{ResNet} and the Fusion-model ($M_{s3}^{\text{ResNet}} + \text{Library}$), respectively.

Classification report for M_{s2}					Classification report for Model M_{s3}				
Type	Precision	Recall	F1-Score	Support	Type	Precision	Recall	F1-Score	Support
AKIEC	0.64	0.53	0.58	34	AKIEC	0.55	0.82	0.66	34
Basal(BCC)	0.76	0.76	0.76	49	Basal(BCC)	0.79	0.78	0.78	49
bkl	0.69	0.69	0.69	109	bkl	0.79	0.69	0.74	109
DF	1.00	0.36	0.53	11	DF	1.00	0.45	0.62	11
NV	0.91	0.96	0.94	811	NV	0.92	0.95	0.94	811
Melanoma	0.58	0.41	0.48	92	Melanoma	0.62	0.46	0.53	92
Vascular	0.85	0.73	0.79	15	Vascular	0.83	1.00	0.91	15
Macro avg	0.78	0.63	0.68	1121	Macro avg	0.79	0.74	0.74	1121
Weighted avg	0.85	0.86	0.85	1121	Weighted avg	0.87	0.87	0.87	1121
Accuracy			0.8582		Accuracy			0.8706	0
Classification report for Model M_{s3} with Library 1									
Type	Precision	Recall	F1-Score	Support					
AKIEC	0.55	0.82	0.66	34					
Basal(BCC)	0.84	0.78	0.81	49					
bkl	0.79	0.68	0.73	109					
DF	1.00	0.45	0.62	11					
NV	0.92	0.96	0.94	811					
Melanoma	0.62	0.47	0.53	92					
Vascular	0.83	1.00	0.91	15					
Macro avg	0.79	0.74	0.74	1121					
Weighted avg	0.87	0.87	0.87	1121					
Accuracy			0.8724	0					

Table 5

Category representation (Library) experiment on validation data for targeting the poor performance layer in fine-tuning EfficientNet M_{s2} .

Classification report for Layer-1 by constructed Library-1					Classification report for Layer-2 by constructed Library-2				
Type	Precision	Recall	F1-Score	Support	Type	Precision	Recall	F1-Score	Support
AKIEC	0.31	0.50	0.38	34	AKIEC	0.33	0.53	0.41	34
Basal(BCC)	0.18	0.29	0.22	49	Basal(BCC)	0.37	0.39	0.38	49
bkl	0.30	0.42	0.35	109	bkl	0.44	0.40	0.42	109
DF	0.11	0.36	0.16	11	DF	0.12	0.18	0.14	11
NV	0.92	0.75	0.83	810	NV	0.88	0.86	0.87	810
Melanoma	0.27	0.36	0.31	92	Melanoma	0.27	0.27	0.27	92
Vascular	0.40	0.27	0.32	15	Vascular	0.33	0.27	0.30	15
Macro avg	0.35	0.42	0.37	1120	macro avg	0.39	0.42	0.40	1120
Weighted avg	0.74	0.65	0.68	1120	Weighted avg	0.73	0.72	0.73	1120
Accuracy			0.65	0	Accuracy			0.72	0
Classification report for Layer-3 by constructed Library-3									
Type	Precision	Recall	F1-Score	Support					
AKIEC	0.78	0.53	0.63	34					
Basal(BCC)	0.64	0.69	0.67	49					
bkl	0.61	0.58	0.59	109					
DF	1.00	0.36	0.53	11					
NV	0.92	0.87	0.90	810					
Melanoma	0.28	0.48	0.36	92					
Vascular	0.80	0.80	0.80	15					
Macro avg	0.72	0.62	0.64	1120					
Weighted avg	0.82	0.79	0.80	1120					
Accuracy			0.79	0					

To further evaluate the quality of deep neural networks (DNNs), we utilize WeightWatcher, an open-source diagnostic tool designed to analyze DNNs without requiring access to training or test data. It is grounded in the Theory of Heavy-Tailed Self-Regularization (SR), which helps determine if and when a specific DNN layer has converged correctly. Unlike traditional tools, WeightWatcher provides unique insights into the performance of individual layers.

When running WeightWatcher, the tool generates a results table, with a key metric being the “ α ” value, which indicates the quality of the layer’s training. Ideally, α values should fall between 2 and 6, signaling that a layer is well-trained. Layers with α values outside this range may require further optimization. The table below presents the analysis results of the trained ResNet18 (M_{s2}^{ResNet}) and EfficientNet (M_{s2}^{EffNet}) models.

The results of the WeightWatcher analysis for the ResNet18 (M_{s2}^{ResNet}) and EfficientNet (M_{s2}^{EffNet}) models are presented in [Tables 7 and 8](#), respectively. From [Table 8](#), we observe that all layers were successfully analyzed. However, five convolutional layers (2, 3, 6, and 7) are marked as undertrained, while ten convolutional layers are marked as overtrained. In [Table 7](#), *layer1.0.conv1* is identified as over-trained with an α value of 1.95, and *net.layer3.0.conv1* is flagged as under-trained with an α value of 6.69.

[Fig. 4](#) illustrates the distribution of WeightWatcher’s layer quality metric α for both models, where the majority of layers have α values between 2.0 and 6.0, indicating that most layers are well trained. However, both models contain under- and over-trained layers. The overall α score for M_{s2}^{ResNet} is 3.47, and for M_{s2}^{EffNet} , it is 3.69, suggesting that both models are generally well trained.

Table 6

The classification report on testing data by M_{s2}^{EffNet} , M_{s3}^{EffNet} and the Fusion-model ($M_{s3}^{\text{EffNet}} + \text{Library}$), respectively.

Classification report for M_{s_2}					Classification report for Model M_{s_3}					
Type	Precision	Recall	F1-Score	Support	Type	Precision	Recall	F1-Score	Support	
AKIEC	0.61	0.65	0.63	34	AKIEC	0.62	0.71	0.66	34	
Basal(BCC)	0.82	0.86	0.84	49	Basal(BCC)	0.84	0.88	0.86	49	
bkl	0.66	0.73	0.70	109	bkl	0.70	0.71	0.70	109	
DF	1.00	0.27	0.43	11	DF	0.56	0.45	0.50	11	
NV	0.96	0.91	0.93	811	NV	0.95	0.93	0.94	811	
Melanoma	0.48	0.66	0.56	92	Melanoma	0.61	0.62	0.61	92	
Vascular	0.88	1.00	0.94	15	Vascular	0.65	1.00	0.79	15	
Macro avg	0.77	0.73	0.72	1121	Macro avg	0.70	0.76	0.72	1121	
Weighted avg	0.87	0.85	0.86	1121	Weighted avg	0.88	0.87	0.87	1121	
Accuracy				0.85	Accuracy				0.87 (0.8724)	0

Classification report for Model M_{s_3} with Library-3				
Type	Precision	Recall	F1-Score	Support
AKIEC	0.62	0.71	0.66	34
Basal(BCC)	0.84	0.88	0.86	49
bkl	0.71	0.72	0.71	109
DF	0.62	0.45	0.53	11
NV	0.95	0.94	0.94	811
Melanoma	0.62	0.62	0.62	92
Vascular	0.65	1.00	0.79	15
Macro avg	0.72	0.76	0.73	1121
Weighted avg	0.88	0.88	0.88	1121
Accuracy			0.88 (0.8760)	0

Table 7

Details analysis on status, α , and warning metrics to trained EfficientNet (M_{s2}^{EffNet}) by WeightWatcher.

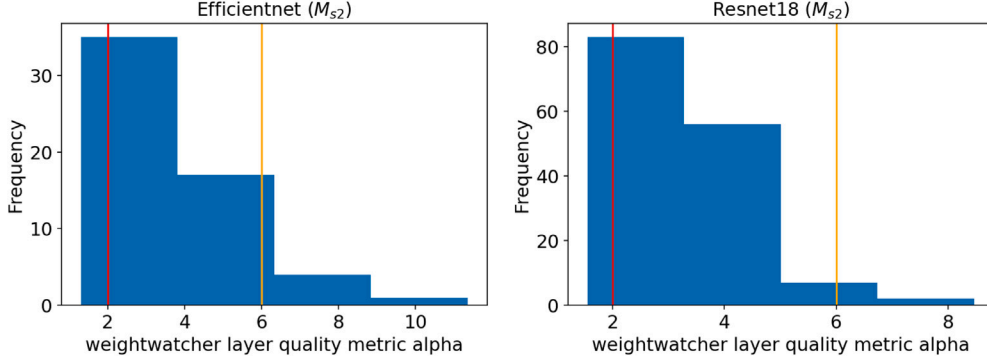
Longname	Alpha	Status	Warning	Longname	Alpha	Status	Warning
net.features.0.0	1.65	tested	over-trained	net.features.5.1.block.2.fc2	3.05	tested	
net.features.1.0.block.2.0	2.20	tested		net.features.5.1.block.3.0	3.78	tested	
net.features.2.0.block.0.0	2.74	tested		net.features.5.2.block.0.0	3.03	tested	
net.features.2.0.block.3.0	6.62	tested	under-trained	net.features.5.2.block.1.0	1.85	tested	over-trained
net.features.2.1.block.0.0	3.10	tested		net.features.5.2.block.2.fc1	3.50	tested	
net.features.2.1.block.3.0	4.16	tested		net.features.5.2.block.2.fc2	3.32	tested	
net.features.3.0.block.0.0	2.79	tested		net.features.5.2.block.3.0	3.62	tested	
net.features.3.0.block.1.0	1.75	tested	over-trained	net.features.6.0.block.0.0	2.74	tested	
net.features.3.0.block.3.0	4.13	tested		net.features.6.0.block.1.0	1.36	tested	over-trained
net.features.3.1.block.0.0	2.23	tested		net.features.6.0.block.2.fc1	1.38	tested	over-trained
net.features.3.1.block.1.0	1.69	tested	over-trained	net.features.6.0.block.2.fc2	1.31	tested	over-trained
net.features.3.1.block.2.fc1	3.78	tested		net.features.6.0.block.3.0	7.90	tested	under-trained
net.features.3.1.block.2.fc2	3.00	tested		net.features.6.1.block.0.0	3.78	tested	
net.features.3.1.block.3.0	6.55	tested	under-trained	net.features.6.1.block.1.0	2.78	tested	
net.features.4.0.block.0.0	5.62	tested		net.features.6.1.block.2.fc1	4.80	tested	
net.features.4.0.block.2.fc1	3.69	tested		net.features.6.1.block.2.fc2	3.63	tested	
net.features.4.0.block.2.fc2	3.81	tested		net.features.6.1.block.3.0	5.01	tested	
net.features.4.0.block.3.0	4.22	tested		net.features.6.2.block.0.0	4.23	tested	
net.features.4.1.block.0.0	4.33	tested		net.features.6.2.block.1.0	2.87	tested	
net.features.4.1.block.2.fc1	3.26	tested		net.features.6.2.block.2.fc1	3.68	tested	
net.features.4.1.block.2.fc2	2.82	tested		net.features.6.2.block.2.fc2	3.71	tested	
net.features.4.1.block.3.0	2.80	tested		net.features.6.2.block.3.0	5.57	tested	
net.features.4.2.block.0.0	2.70	tested		net.features.6.3.block.0.0	3.94	tested	
net.features.4.2.block.2.fc1	3.51	tested		net.features.6.3.block.1.0	2.88	tested	
net.features.4.2.block.2.fc2	2.84	tested		net.features.6.3.block.2.fc1	4.29	tested	
net.features.4.2.block.3.0	3.34	tested		net.features.6.3.block.2.fc2	3.92	tested	
net.features.5.0.block.0.0	2.88	tested		net.features.6.3.block.3.0	5.93	tested	
net.features.5.0.block.1.0	1.58	tested	over-trained	net.features.7.0.block.0.0	3.23	tested	
net.features.5.0.block.2.fc1	4.18	tested		net.features.7.0.block.2.fc1	11.36	tested	under-trained
net.features.5.0.block.2.fc2	5.66	tested		net.features.7.0.block.2.fc2	3.08	tested	
net.features.5.0.block.3.0	4.00	tested		net.features.7.0.block.3.0	7.45	tested	under-trained
net.features.5.1.block.0.0	3.35	tested		net.features.8.0	4.58	tested	
net.features.5.1.block.1.0	1.82	tested	over-trained	net.classifier.1	3.22	tested	
net.features.5.1.block.2.fc1	3.68	tested					

The weightwatcher layer quality evaluation has two key issues to identify poorly performing layers (see Table 9): (1) the overall evaluation uses randomized singular value decomposition (SVD), which causes slight variation in results between different runs, as demonstrated in Section 4.6 (Table 11) with varying under-trained layer

results; and (2) relying solely on the under-trained and over-trained evaluations has shown some tested (success evaluated) to improve model performance. In Section 4.6, we show that retraining based only on the flagged convolutional layers did not significantly enhance the model's performance.

Table 8Details analysis on status, rank_loss (rl), α , and warning metrics to trained ResNet (M_{s2}^{ResNet}) by WeightWatcher.

Longname	Alpha	rl	Status	Warning	Longname	Alpha	rl	Status	Warning
net.conv1	2.12	0	tested		net.layer3.0.conv2	4.87	0	tested	
net.layer1.0.conv1	1.95	72	tested	over-trained	net.layer3.0.downsample.0	3.13	0	tested	
net.layer1.0.conv2	2.47	0	tested		net.layer3.1.conv1	3.73	2	tested	
net.layer1.1.conv1	2.32	0	tested		net.layer3.1.conv2	3.02	0	tested	
net.layer1.1.conv2	2.62	0	tested		net.layer4.0.conv1	3.76	0	tested	
net.layer2.0.conv1	4.30	0	tested		net.layer4.0.conv2	3.37	0	tested	
net.layer2.0.conv2	2.85	0	tested		net.layer4.0.downsample.0	2.20	0	tested	
net.layer2.0.downsample.0	3.81	0	tested		net.layer4.1.conv1	3.58	2	tested	
net.layer2.1.conv1	2.73	0	tested		net.layer4.1.conv2	4.41	2	tested	
net.layer2.1.conv2	3.16	0	tested		net.fc	5.75	0	tested	
net.layer3.0.conv1	6.69	0	tested	under-trained					

**Fig. 4.** WeightWatcher layer quality metric α evaluation diagram for ResNet18 (M_{s2}^{ResNet}) and EfficientNet (M_{s2}^{EffNet}) models.**Table 9**

List of characteristics on two methods in poorly layer identification of trained model.

Method	Targets	Require data	Task specific	Metric	Theory	Valid range
The proposed	Feature data	Yes	Yes	Accuracy	Sparse representation	acc 0–1
WeightWatcher	Model's weights	No	No	alpha value	Heavy-Tailed SR	alpha 2–6
Evaluation unit		inference		Performance in SKIN cancer data		
The proposed	Each layer/blocks	Yes (comparison between layer in one model)			Reasonable	
WeightWatcher	Convolutional layer	Yes (comparison between models)			–	

4.5. Interpretability in confusion report and category representation

The proposed category representation not only aids in evaluating and identifying underperforming layers, allowing targeted fine-tuning to enhance the model, but also assists in interpreting feature representations and predicting classification results.

4.5.1. Confusion report interpretability

The well-trained category representation, later constructed as the *Library*, can interpret new features derived from test images of unknown types. From a medical standpoint, skin cancer types are rigorously defined according to specialist guidelines on abnormal skin cell behavior. Structurally, different cases of the same skin cancer type share common features aligned with these regulations. Consequently, images of the same type of skin cancer tend to produce homogeneous features when processed through the model pipeline.

UMAP, a manifold learning technique, is employed to simulate and learn the intrinsic low-dimensional topology of the stacked category representations, effectively clustering similar data points while separating distinct ones. We use UMAP to project the *Library* into a two-dimensional space, where each skin cancer type is represented by a unique color, as shown in Fig. 5(a). The projection forms a Y-shaped (trident-like) scatterplot, with a few representative test data points displayed as squares. In this figure, seven distinct colors represent seven different skin cancer types. Notably, the data points corresponding to the NV type are clearly separated from other groups, facilitating

easier identification. According to the classification report, the NV type achieved a precision score greater than 0.95. However, some data points of the MEL type overlap with NV and BKL types, making MEL harder to identify and contributing to misclassification into NV, as seen in the confusion matrix $C_{\text{EfficientNET}}^{M_{s3}}$, where 19 errors appear in the NV type out of 34 misclassified instances in MEL test data.

The AKIEC data points, represented in red, form a small cluster near BCC and a few MEL and BKL points. This overlap causes occasional misclassifications of the BCC and BKL cases in AKIEC, as reflected in $C_{\text{EfficientNET}}^{M_{s3}}$. Most categories show some misclassification into the AKIEC type.

Furthermore, Fig. 5(b)–(d) presents the 2D mapping of the *Library* representation, considering localization data. This mapping reflects the distribution of different skin cancer types in specific body regions, as indicated by the pie chart in Fig. 2. For example, in the localization of the face, we observe that the Dermatofibroma (DF) type does not appear in this region. As a result, we map all category representations from the *Library* except for DF, as shown in Fig. 5(b). The Y-shaped scatter plot features six distinct color clouds, representing six skin cancer types, with square-shaped data points for BKL, MEL, and AKIEC well-distributed within their respective clouds, highlighting their characteristic features.

In Fig. 5(c), we map the data for the foot localization, where AKIEC and VASC skin cancer types have no recorded cases. In Fig. 5(b), the MEL type forms a Y-shaped scatter in three different locations, primarily in the center of the plot, with a few points overlapping with

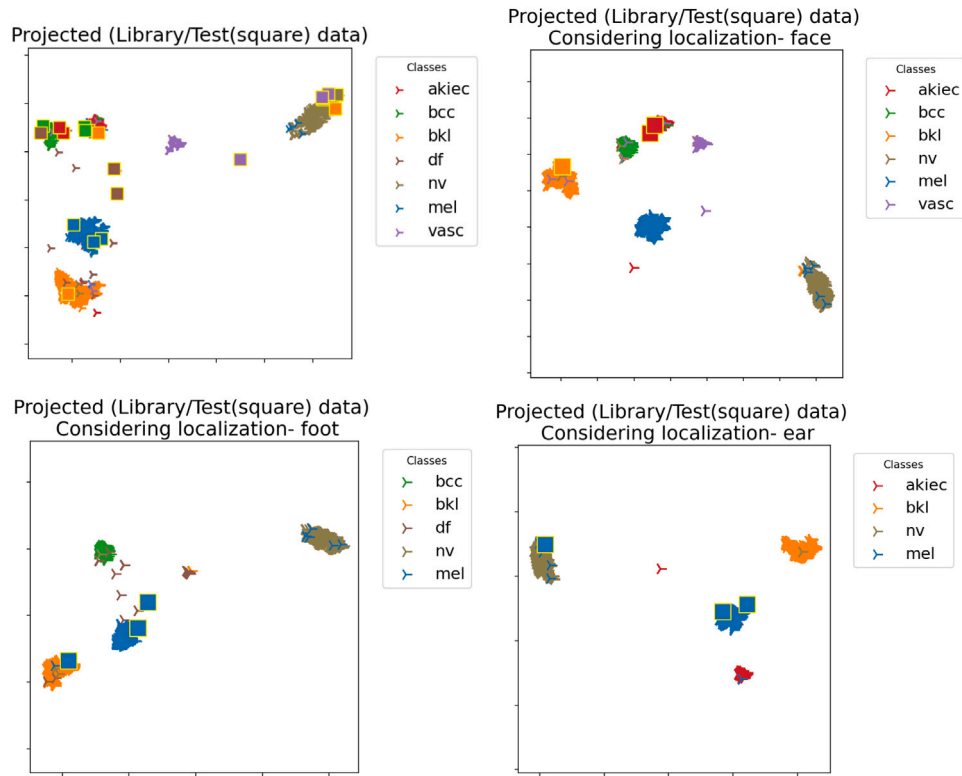


Fig. 5. Visualization of category representation mapping, where colored Y-shapes represent the data clouds of the Library, with colors corresponding to different skin cancer types. The square-shaped data points indicate mapped test images and their coordinates denote the classification label candidates. For detailed interpretability, refer to Figs. 7 and 8.

BKL and NV data clouds, which leads to a misclassification of MEL in one instance.

Fig. 5(d) illustrates the types of skin cancer localized to the ear, where only AKIEC, BKL, NV, and MEL are present. If we focus solely on these types, it becomes evident that MEL dominates the classification task, making it easier to identify the top two closest matches for any new skin image. For example, a square-shaped blue data point in the top right corner of the NV cloud is surrounded only by MEL and NV data. This enables us to narrow down the classification options and improve precision.

In summary, the category representation provides valuable information on the effectiveness of the learned characteristics within the neural network model and helps to predict the classifications and top- n (where n depending on the data clouds of the closest category) candidate skin cancer types. This can be very beneficial in supporting medical decision making and treatment planning.

4.5.2. Comparison of interpretability results

Various open-source methods have been developed to enhance the interpretability of the model, allowing researchers and developers to easily identify the characteristics that influence the output of a model.

In this section, we experiment with Captum (meaning “comprehension” in Latin) <https://captum.ai/>, and SHAP (SHapley Additive Explanations) <https://shap.readthedocs.io/en/latest/#>, two popular methods for interpreting the results of models M_{53}^{ResNet} and M_{53}^{EffNet} . SHAP is a game-theoretic approach that explains the output of any machine learning model, while Captum, an open-source library built on PyTorch, offers advanced interpretability algorithms such as Integrated Gradients. Due to computational limitations, we specifically employed DeepLIFT and Saliency from the Captum library. DeepLIFT assigns significance to each input pixel by comparing the differences between the output and a reference relative to the differences between the input and the reference. Saliency, a baseline method for input attribution, returns gradients with respect to inputs, providing their absolute values by default.

Fig. 6 illustrates the interpretability results of DeepLIFT and Saliency. Although color changes and dot points indicate the importance of how the model transforms the image, this information provides little practical insight for medical decision-making. Furthermore, the results seem overly complex and do not clearly indicate whether the model performs well. Fig. 7(a) presents the SHAP results, where the image pixels are highlighted according to the SHAP values, reflecting the importance of the individual pixels for the classification decision. However, in skin cancer images, noise is scattered across the image, making it difficult to isolate the exact areas that contribute to specific cancer classifications.

Regarding the proposed approach, the interpretability results are shown in Figs. 7(b) and 8(b), where data clouds are color-grouped for each type of cancer. In Fig. 8(b), it is evident that the model performs worse in localizing cancer in the *face* area compared to *ear*, as many scattered data points overlap with different groups, making it difficult to classify the cancer type according to nearest neighbors correctly. Additionally, we can observe the top-2 classification candidates. For example, in Fig. 8(b), No. 1 and No. 2 are assigned to different groups, with No. 1 (orange) mapped to the orange group but with a few blue Y-shaped data points. This suggests that the two main possible classifications are BKL and MEL. In the bottom-right corner, the olive-colored group, representing NV-type cancer, overlaps with a few MEL and BKL data points. This could signal a potential classification error between the MEL, BKL, and NV cancer types.

We can also infer that the classification of the AKIEC and NV types of skin cancer is more likely to be correct than that of AKIEC and BCC, as evidenced by the greater distance between their groups in the mapped data.

Captum and SHAP interpreters focus on explaining a model’s internal mechanisms, the proposed method aids in interpreting classification decisions, providing valuable insights for medical treatment decisions. Table 10 reviews the characteristics of the interpretability methods applied to medical datasets, where $\times$ and $\checkmark$ denote the applicability

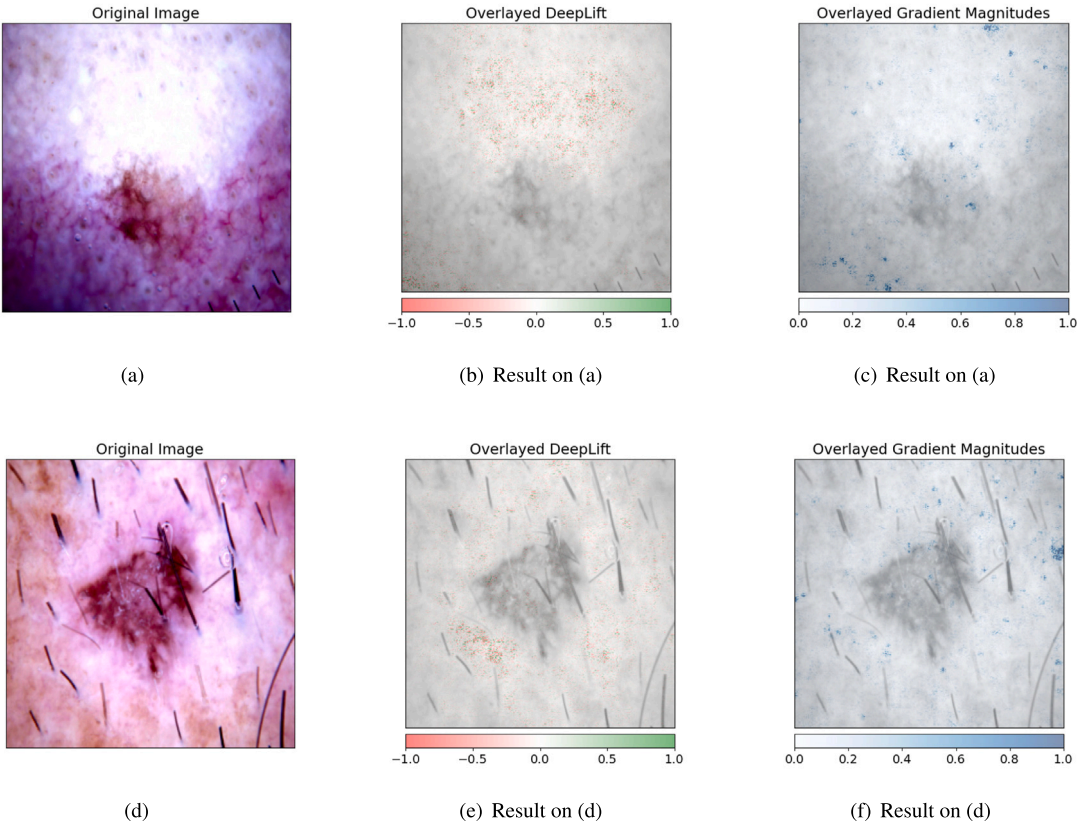


Fig. 6. Results of DeepLIFT and Saliency interpretability on two images: (a) and (d) show the original images, while (b)–(c) and (e)–(f) display the corresponding DeepLIFT and Saliency interpretability results for images (a) and (d), respectively.

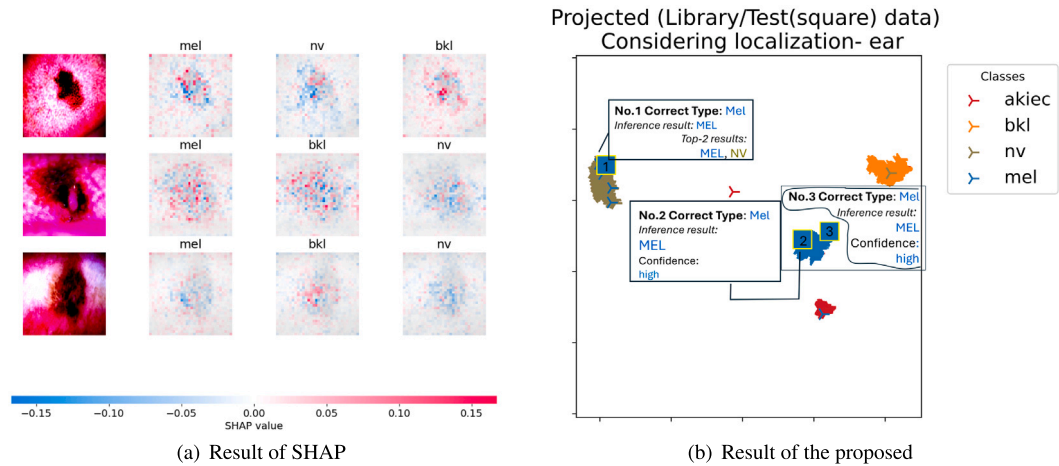


Fig. 7. Comparison of interpretability results between SHAP and Category representation. SHAP was executed on a GPU100 provided by Kaggle, using 21 references due to memory allocation limitations.

Table 10
A review of the characteristics of interpretability methods applied to medical datasets, where × and ✓ denote the applicability to the relevant attributes.

Model	Memory require	Top-n Inference	Calculating	Error inference	Inter-class Relation	Interpretability in medical
SHAP	High	×	SHAP value	×	×	Low
Saliency	Low	×	Gradients	×	×	Low
DeepLIFT	Middle	×	Attributes Significance	×	×	Low
Our	Low	✓	Intrinsic Category feature	✓	✓	Acceptable

SHAP: <https://shap.readthedocs.io/>; Saliency and DeepLIFT are from *Captum.ai*.

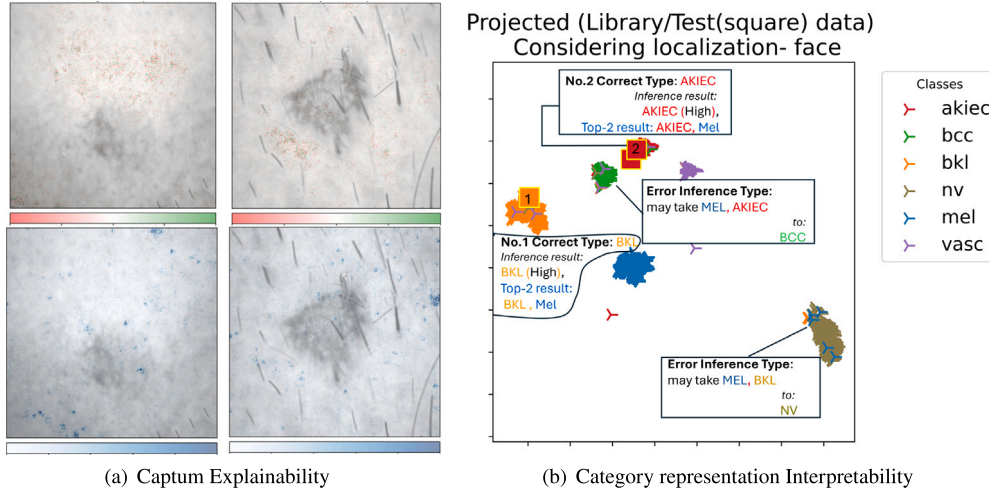


Fig. 8. Comparison of interpretability results: Captum emphasizes understanding a model's internal mechanics, while our Category representation technique explains the rationale behind the model's decisions.

to the relevant attributes. Medical data often exhibit subtle and slight variations across datasets, making interpretability crucial. Therefore, a high level of explainability in decision-making is necessary to support effective medical treatment.

Furthermore, Fig. 9 illustrates the top- n predictions. (1) Subfigures (a)–(c) show cases where the top-1 prediction is correct. (2) Subfigures (d)–(f) demonstrate instances where the top-2 prediction includes the correct answer. (3) The last three subfigures represent cases where the prediction of the top 3 includes the correct result.

The prediction accuracy of the top-3 reaches 98.21%, which is highly significant for medical decision making. Moreover, the figure provides insights into how top- n predictions influence the final decision.

For example:

1. In Fig. 9(a), the model predicts the cancer type as 'nv' (correct) with the second choice being 'vasc'. The probability of 'vasc' is 0.23, which is much lower than 0.5, giving higher confidence to the 'nv' prediction.
2. In Fig. 9(d), the most probable types are 'nv' and 'bkl' with relative scores of 1.0 and 0.86, respectively. The new data (orange square) is located in the 'nv' group area, leading to a higher confidence in 'nv' as the prediction, even though the true type is 'bkl'. This highlights the importance of considering top- n predictions rather than relying solely on the top-1 result.
3. Similarly, in Fig. 9(g), the model's top prediction is 'mel' (incorrect). However, the top-3 predictions ('mel', 'bkl', and 'akiec') have high scores of 1.0, 0.93, and 0.85, respectively. The visualization shows that the new data (red square) is situated between the 'mel', 'bkl', and 'akiec' groups, making it reasonable to consider the top-3 predictions. The true type is 'akiec', reinforcing the need to analyze beyond the prediction of top-1.

This observation underscores the value of top- n predictions, especially in challenging cases, to support more informed and accurate decisions.

In conclusion, the proposed method not only interprets the model's decisions, but also gives insight into the likelihood of classification errors, making it particularly valuable for clinicians when dealing with ambiguous cases. This can directly aid in choosing follow-up tests or treatments, which is less apparent in pixel-attribution-based methods. In contrast to the complexity of SHAP and Captum outputs, the visual clustering of our method simplifies the interpretation for clinicians, allowing them to visually and intuitively assess the model's confidence

across different cancer types. Ultimately, this interpretability framework can guide clinicians in making better-informed decisions, such as whether additional diagnostic tests are necessary, thereby reducing the risk of false positives or misdiagnoses in high-stakes environments like cancer detection.

4.6. WeightWatcher and baseline results

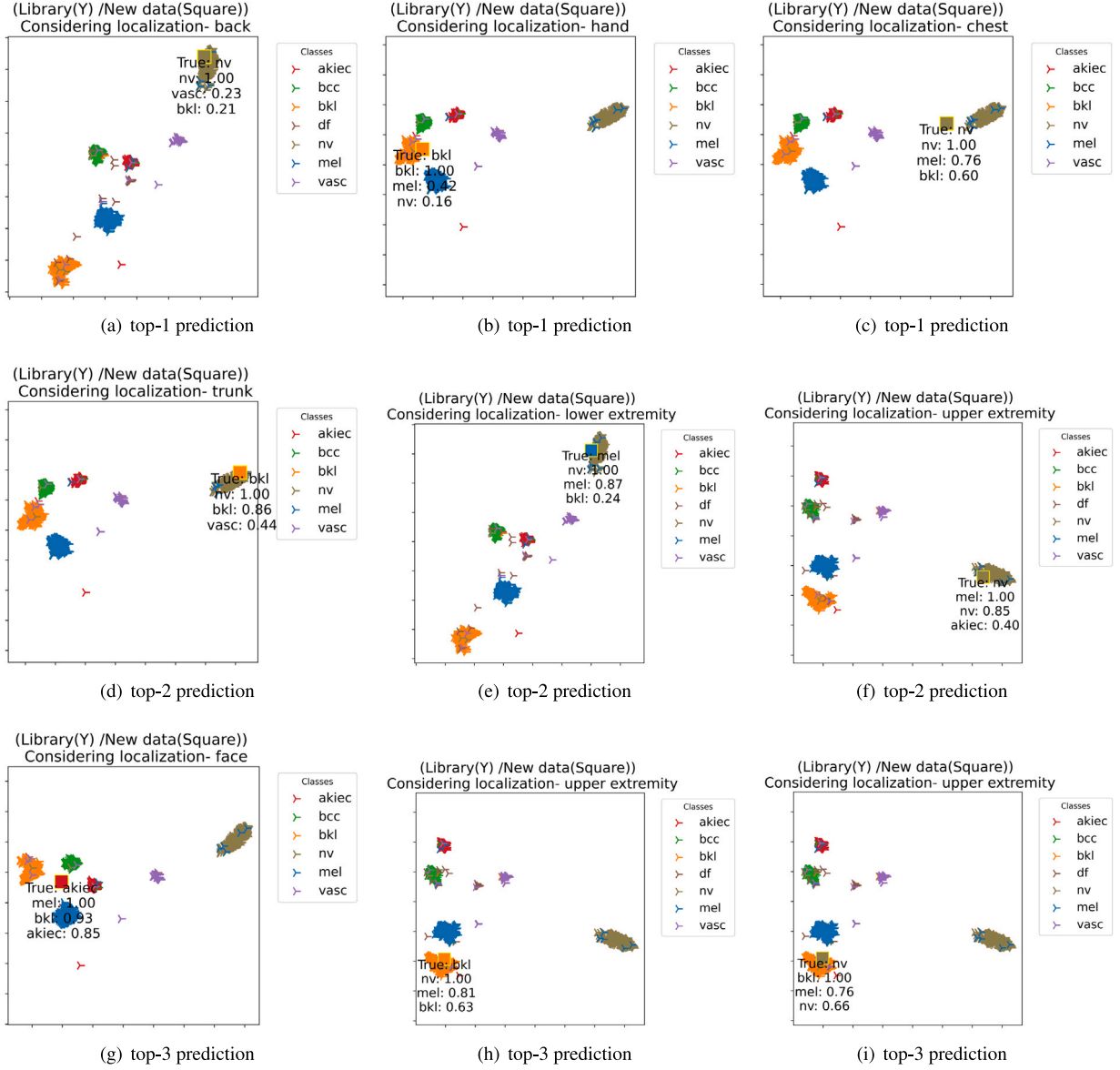
In the previous subsections, we presented the performance of the pre-trained model and compared it with the baseline without external references. Here, we provide the results of the pre-trained model with fine-tuning for reference. Additionally, in the interpretability and model improvement section, we introduced WeightWatcher without detailing the process, which we now provide here.

4.6.1. WeightWatcher assist in model improvement

As discussed in the experiment section, applying WeightWatcher analysis results during the retraining process did not lead to an overall improvement in model performance. This section provides a detailed explanation of that application process.

WeightWatcher utilizes the Randomized Full SVD method to calculate eigenvalues, which can result in slight variations across multiple runs. To account for this, we performed the analysis 10 times, setting the *min_evals* parameter to 10, and generated the results shown in Table 11. Taking EfficientNet as an example, Table 11 indicates that the *features.6.0* block.2 has two over-trained convolutional layers, while five operations in the *features.[2, 3, 6, 7]* layers are under-trained. Based on these observations, we constructed a reference model M_2 (model2).

1. Construct a model with pre-trained weights default, *model1* = *models.efficientnet_b0(weights = 'DEFAULT')*
2. Construct and Load a model2 with custom weights as below.
 - (a) first we initialize with no pre-trained weights;
model2 = *models.efficientnet_b0(weights = None)*,
 - (b) Then we load it as our pretrained M_{s2}^{EffNet} model,
model2.load_state_dict(torch.load(our M_{s2}^{EffNet})).
3. Replace the weights of model2 in the layers flagged with an over-trained warning:
model2.load_partial_state_dict(model1.state_dict(net.features.6.0.block.2.fc1, net.features.6.0.block.2.fc2)),
 - (a) Note: The two layers, 'net.features.6.0.block.2.fc1' and 'net.features.6.0.block.2.fc2', are identified in Table 11.



Prediction accuracy	Top-1	Top-2	Top-3
Proposed Model	87.24	94.83	98.21

Fig. 9. Visualization of top-n predictions with category representation. Top-1 accuracy indicates that the prediction with the highest probability is correct. Top-2 accuracy signifies that at least one of the top two predictions is correct. Similarly, top-3 accuracy means that at least one of the top three predictions is correct.

(b) Freeze all the parameters of model2 except for these two layers, making only their parameters tunable.

- For layers flagged as under-trained, the poorly performing neurons include 'net.features.2.0.block.3.0, 3.1.block.3.0, 6.0.block.3.0, 7.0.block.2.fc1, 7.0.block.3.0'. We will set these layers in model2 as tunable, allowing their gradients to be updated during training.
- Re-tune model2 with the same experiment setting as in tuning M_{s2} , then we get model $M_{watcher}$.

We evaluated the model M_{s2} as the baseline and compared its performance with the WeightWatcher-tuned model $M_{watcher}$. The following results were obtained:

The average accuracy of $M_{watcher}$ on the test data was 0.8591, calculated as $\{(86.08 + 85.37 + 85.10 + 86.44 + 85.37 + 86.17 + 86.80$

$+ 85.01 + 86.08 + 86.17)/10\}$, with a highest accuracy of 0.8680 and a lowest accuracy of 0.8501. In comparison, the baseline accuracy of model M_{s2} was 85.91%. Overall, the WeightWatcher-assisted retraining process did not result in performance improvement, indicating that further strategies are required to effectively leverage WeightWatcher's analysis.

4.6.2. Baseline model performance

As discussed in the Introduction, there is no standard protocol for experiment settings, even when using the same skin cancer dataset. Variables such as the number of images, inclusion of duplicates, training/validation/test split ratios, and network training costs influence the evaluation outcomes, making it challenging to conduct fair comparisons with the existing literature. However, we can still derive insights from some results as a reference. Table 12 presents the classification

Table 11

Details analysis on status, rank_loss (rl), α , and warning metrics to trained ResNet and EfficientNet ($M_{s_2}^{\text{ResNet}}$, $M_{s_2}^{\text{EffNet}}$) by WeightWatcher with $\min_evals$ equal to 10.

Longname	Alpha	Status	Warning	Longname	Alpha	Status	Warning
ResNet				EfficientNet			
net.layer1.0.conv1	1.72	tested	over-trained	net.features.6.0.block.2.fc1	1.38	tested	over-trained
net.layer1.0.conv1	1.61	tested	over-trained	net.features.6.0.block.2.fc2	1.31	tested	over-trained
net.layer1.0.conv1	1.85	tested	over-trained	net.features.2.0.block.3.0	6.62	tested	under-trained
net.layer1.0.conv1	1.55	tested	over-trained	net.features.3.1.block.3.0	6.55	tested	under-trained
net.layer1.1.conv1	1.96	tested	over-trained	net.features.6.0.block.3.0	7.90	tested	under-trained
net.layer1.1.conv1	1.98	tested	over-trained	net.features.7.0.block.2.fc1	11.36	tested	under-trained
net.layer1.1.conv1	1.77	tested	over-trained	net.features.7.0.block.3.0	7.45	tested	under-trained
net.layer1.1.conv1	1.75	tested	over-trained				
net.layer2.0.conv1	1.82	tested	over-trained				
net.layer2.0.conv2	1.63	tested	over-trained				
net.layer2.0.conv2	1.83	tested	over-trained				
net.layer2.1.conv1	1.96	tested	over-trained				
net.layer1.1.conv2	6.89	tested	under-trained				
net.layer3.0.conv1	6.27	tested	under-trained				
net.layer3.0.conv1	8.47	tested	under-trained				

Table 12

Performance of various baseline pre-trained model on Accuracy, Recall.

Source: Results are taken from [20], the parameter (in Million, termed as Parames) and Input image (size) information is taken from [33].

Model	EffNet B0	EffNet B1	EffNet B2	EffNet B3	EffNet B4	EffNet B5	EffNet B6	EffNet B7
Accuracy	0.8302	0.8369	0.8395	0.8390	0.8791	0.8762	0.8536	0.8552
Recall	0.83	0.84	0.84	0.84	0.88	0.88	0.85	0.86
Parameter	4M	6M	7M	10M	17M	28M	40M	64M
Input Image	224 × 224	240 × 240	260 × 260	300 × 300	380 × 380	456 × 456	528 × 528	600 × 600
Paper	Model	Parames	Accuracy	Paper	Model	Parames	Accuracy	
Ali 2022 [20]	EfficientNetB4	17M	87.91	pant 2022 [21]	X-R50	48M	97.8	
Shahin 2021 [22]	DCNN	–	91.93	Duyen 2021 [23]	ResNet50	25M	93	
Dibaloke 2023 [24]	DCENSnet	–	99.53	Gomathi 2023 [25]	DODL	2.6M	98.76	
Ling 2024 [29]	ResNet50	25M	90.71	DMSFF 2024[28]	ResNet50	25M	87.2	
Omneya 2024 [27]	Inception	13.6M	97.2	Carolin 2024 [26]	ViT	22M	91.57	
Measure Accuracy	Top-1	Top-2	Top-3					
Proposed Model	87.24	94.83	98.21		Resnet18	11M		

ViT: Vision Transformer; ‘–’ indicates cannot reach the source; Model is baseline model.

As shown in Table 1, due to non-uniform experimental settings, performance comparisons are challenging.

This table references results that show the baseline performance under our experimental settings.

results of the eight pre-trained EfficientNets (B0–B7) models on the HAM10000 dataset, using a pre-processing pipeline that includes duplicates in the training set. From Table 12, we observe that the baseline result achieved with EfficientNet-B0 (the model we used) is 0.8302, while our fusion model achieves an accuracy of 0.8760 in the test data.

4.6.3. Complexity and time cost

For a time-cost comparison, we recorded the run time of each step as a reference. The experiments were carried out on a Kaggle platform using four T4 GPUs. The initial fine-tuning of the baseline model on the entire dataset took 346.127 s. The process of identifying nonactive neurons took 86.19 s, followed by fine-tuning the underperforming layer, which required 269.336 s.

For interpretability, SHAP’s explainability in three images with 20 samples of parameter took 46.12 s, whereas generating more reliable results with 200 samples took 210.879 s. The Saliency method from the Captum library required 0.5809 s, but the IntegratedGradients with NoiseTunnel could not be completed due to memory allocation issues on the platform.

The Library category representation interpretability was performed on Google Colab using a CPU, with a runtime of 32.568 s (due to UMAP errors on the Kaggle platform). In general, our proposed methods demonstrated fast processing times in all stages of the workflow.

4.7. Scalability and deployment

We review the potential trade-offs between interpretability, scalability, and computational overhead as follows.

From the deployment point of view, the framework consists of two components: model creation and interpretability. Since only a training and validation dataset are required, retraining and library creation occur in the backend, while medical facilities use a pre-trained model for inference and interpretability analysis.

The EfficientNet model (15.8 MB) and category representation library (3.64 MB) are lightweight, with the library available <https://github.com/lingping-fuzzy/cancer-model>. Computational demands, including UMAP-based visualization, remain feasible for medical facilities.

From the scalability point of view, The model requires fine-tuning when new databases become available to ensure unbiased outcomes. As it does not support continuous learning, periodic retraining is necessary for updates.

5. Conclusion

This paper presents an interpretability-assisted framework for improving the performance of pre-trained models, with a specific focus on medical image classification tasks. By incorporating category representation and optimizing computation through fast aggregation techniques, we significantly reduce resource costs while enhancing model generalization and applicability. Additionally, our approach identifies inactive neurons, allowing for targeted model improvements. Experimental results on skin cancer data, along with comparisons with the WeightWatcher tool, demonstrate that our method achieves higher accuracy and better interpretability. These advancements not only improve classification performance but also provide valuable insights

for medical decision-making, ultimately contributing to more effective and accessible AI-driven healthcare solutions.

In addition, we conducted experiments using the WeightWatcher tool to identify underperforming layers. The results suggest that a more sophisticated strategy may be required to fully leverage WeightWatcher's analysis for model improvement. In our experiments on skin cancer data, with a few epochs of fine-tuning in active neurons, the model achieved a 2.5% improvement in classification performance in the best case. Furthermore, the projection of category representations aligned well with the classification reports, providing reasonable interpretability.

Limitation and Future work

Despite the promising results of the proposed methods, there are several limitations related to interpretability and model performance.

1. The category representation technique is designed to be independent of both the baseline model and dataset, as its interpretability focuses on the model's decisions and confidence in its performance. This flexibility allows the representation to be applied to any baseline model and image dataset. However, in the current framework, the category representation learned from models like ResNet and EfficientNet has shown insufficient feature learning. Furthermore, the framework does not account for metadata, which limits both the performance and the interpretability of medical-related signs and symptom inference. Future work will focus on improving the representation of features and integrating metadata to improve the explanation of critical medical indicators.
2. The interpretability based on category representation provides a general overview of the model's performance, such as decision confidence, top-n classifications, or potential error sources. However, it lacks specific information on decision-making elements. This approach does not capture critical medical indicators or biomarkers that are essential in medical decision making. For example, in the diagnosis of melanoma, distinguishing signs such as a large brownish spot with darker speckles or a mole that changes in color, size, or texture are crucial to accurate classification. However, such medical signs are not included in the current interpretability framework. In future work, our goal is to focus on identifying and integrating these key medical biomarkers to enhance interpretability and directly inform classification decisions.
3. Additionally, the category representation identifies nonactive neurons in pre-trained models, which helps detect poorly trained layers. However, if each layer performs consistently with only minor variations, the category representation may fail to pinpoint neurons for improvement, and consequently may not enhance model performance. For research goals focused on further increasing the classification accuracy, additional strategies and improvements are required.

Specifically, while the model cannot directly classify new skin types, the interpretability visualization can highlight instances where classification confidence is low due to significant embedding differences. This enables physicians to recognize cases that warrant additional examination for more precise diagnosis.

CRedit authorship contribution statement

Lingping Kong: Writing – original draft, Software, Methodology. **Juan D. Velasquez:** Writing – review & editing, Validation, Data curation. **Václav Snášel:** Supervision, Methodology, Conceptualization. **Millie Pant:** Visualization, Validation, Conceptualization. **Jeng-Shyang Pan:** Visualization, Formal analysis, Conceptualization. **Jana Nowakova:** Visualization, Validation, Formal analysis.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors gratefully acknowledge financial support CZ.02.01.01/00/22 008/0004590 by the Czech Republic Ministry of Education, Youth, and Sports in the project “Robotics and Advanced Industrial Production” (ROBOPROX) and the financial support ANID Fondecyt 1231122, ANID PIA/PUENTE AFB230002, Chile. European Union under the REFRESH – number CZ.10.03.01/00/22_/0000 048 via the Operational Programme Just Transition; CLARA project is funded by the European Union's Horizon Europe research and innovation programme No. 101136607, supported in part by the Chunhui Project Foundation of the Education Department of China under Grant HZKY20220487

Data availability

public data, download link is provided in the paper.

References

- [1] Leli Zeng, BH Jaswanth Gowda, Mohammed Gulzar Ahmed, Mohammed AS Abourehab, Zhe-Sheng Chen, Changhua Zhang, Jia Li, Prashant Kesharwani, Advancements in nanoparticle-based treatment approaches for skin cancer therapy, *Mol. Cancer* 22 (1) (2023) 10.
- [2] Fayadh Alenezi, Ammar Armghan, Kemal Polat, A multi-stage melanoma recognition framework with deep residual neural network and hyperparameter optimization-based decision support in dermoscopy images, *Expert Syst. Appl.* 215 (2023) 119352.
- [3] Maryam Naqvi, Syed Qasim Gilani, Tehreem Syed, Oge Marques, Hee-Cheol Kim, Skin cancer detection using deep learning—a review, *Diagnostics* 13 (11) (2023) 1911.
- [4] José María Rodríguez Corral, Javier Civit-Masot, Francisco Luna-Perejón, Ignacio Díaz-Cano, Arturo Morgado-Estévez, Manuel Domínguez-Morales, Energy efficiency in edge TPU vs. embedded GPU for computer-aided medical imaging segmentation and classification, *Eng. Appl. Artif. Intell.* 127 (2024) 107298.
- [5] Wolfgang Birkfellner, *Applied Medical Image Processing: a Basic Course*, CRC Press, 2024.
- [6] Fabio Garcea, Alessio Serra, Fabrizio Lamberti, Lia Morra, Data augmentation for medical imaging: A systematic literature review, *Comput. Biol. Med.* 152 (2023) 106391.
- [7] Bangfeng Wang, Yiwei Li, Mengfan Zhou, Yulong Han, Mingyu Zhang, Zhaolong Gao, Zetai Liu, Peng Chen, Wei Du, Xingcai Zhang, et al., Smartphone-based platforms implementing microfluidic detection with image-based artificial intelligence, *Nat. Commun.* 14 (1) (2023) 1–18.
- [8] Yukun Zhou, Mark A Chia, Siegfried K Wagner, Murat S Ayhan, Dominic J Williamson, Robert R Struyven, Timing Liu, Moucheng Xu, Mateo G Lozano, Peter Woodward-Court, et al., A foundation model for generalizable disease detection from retinal images, *Nature* 622 (7981) (2023) 156–163.
- [9] Rajpurkar Pranav, Irvin Jeremy, Zhu Kaylie, Yang Brandon, Mehta Hershel, CheXNet: Radiologist-level pneumonia detection on chest X-Rays with deep learning, 5225, 2017, <https://arxiv.org/abs/1711.05225v1>.
- [10] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, Piotr Dollár, Focal loss for dense object detection, *IEEE Trans. Pattern Anal. Mach. Intell.* 42 (2) (2020) 318–327.
- [11] T-YL.P.G. Ross, GKHP Dollár, Focal loss for dense object detection, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 2980–2988.
- [12] Fausto Milletari, Nassir Navab, Seyed-Ahmad Ahmadi, V-net: Fully convolutional neural networks for volumetric medical image segmentation, in: *2016 Fourth International Conference on 3D Vision (3DV)*, Ieee, 2016, pp. 565–571.
- [13] Sihong Chen, Kai Ma, Yefeng Zheng, Med3d: Transfer learning for 3d medical image analysis, 2019, arXiv preprint [arXiv:1904.00625](https://arxiv.org/abs/1904.00625).
- [14] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, Kilian Q Weinberger, Densely connected convolutional networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 4700–4708.
- [15] Jinhuk Lee, Wonjin Yoon, Sungdong Kim, Donghyeon Kim, Sunkyu Kim, Chan Ho So, Jaewoo Kang, BioBERT: a pre-trained biomedical language representation model for biomedical text mining, *Bioinformatics* 36 (4) (2020) 1234–1240.

- [16] Tomoki Uemura, Janne J Näppi, Toru Hironaka, Hyoungeop Kim, Hiroyuki Yoshida, Comparative performance of 3D-DenseNet, 3D-ResNet, and 3D-VGG models in polyp detection for CT colonography, in: *Medical Imaging 2020: Computer-Aided Diagnosis*, Vol. 11314, SPIE, 2020, pp. 736–741.
- [17] Fabian Isensee, Paul F Jaeger, Simon AA Kohl, Jens Petersen, Klaus H Maier-Hein, Nnu-net: a self-configuring method for deep learning-based biomedical image segmentation, *Nature Methods* 18 (2) (2021) 203–211.
- [18] Essam H Houssein, Doaa A Abdelkareem, Gang Hu, Mohamed Abdel Hameed, Ibrahim A Ibrahim, Mina Younan, An effective multiclass skin cancer classification approach based on deep convolutional neural network, *Clust. Comput.* (2024) 1–21.
- [19] Fayadh Alenezi, Ammar Armghan, Kemal Polat, Wavelet transform based deep residual neural network and ReLU based extreme learning machine for skin lesion classification, *Expert Syst. Appl.* 213 (2023) 119064.
- [20] Karar Ali, Zaffar Ahmed Shaikh, Abdullah Ayub Khan, Asif Ali Laghari, Multiclass skin cancer classification using EfficientNets—a first step towards preventing skin cancer, *Neurosci. Inform.* 2 (4) (2022) 100034.
- [21] Alavikunhu Panthakkan, SM Anzar, Sangeetha Jamal, Wathiq Mansoor, Concatenated Xception-ResNet50—A novel hybrid approach for accurate skin cancer prediction, *Comput. Biol. Med.* 150 (2022) 106170.
- [22] Md Shahin Ali, Md Sipon Miah, Jahurul Haque, Md Mahbubur Rahman, Md Khairul Islam, An enhanced technique of skin cancer classification using deep convolutional neural network with transfer learning models, *Mach. Learn. Appl.* 5 (2021) 100036.
- [23] Duyen N.T. Le, Hieu X. Le, Lua T. Ngo, Hoan T. Ngo, Transfer learning with class-weighted and focal loss function for automatic skin cancer classification, *Artif. Intell. (cs.AI)* (2020) arXiv:2009.05977.
- [24] Dibalohe Chanda, Md Saif Hassan Onim, Hussain Nyeem, Tareque Bashir Ovi, Sauda Suara Naba, Dcensnet: A new deep convolutional ensemble network for skin cancer classification, *Biomed. Signal Process. Control.* 89 (2024) 105757.
- [25] E. Gomathi, M. Jayasheela, M. Thamarai, M. Geetha, Skin cancer detection using dual optimization based deep learning network, *Biomed. Signal Process. Control.* 84 (2023) 104968.
- [26] Carolin Flösdorf, Justin Engelker, Igor Keller, Nicolas Mohr, Skin cancer detection utilizing deep learning: Classification of skin lesion images using a vision transformer, 2024, arXiv preprint arXiv:2407.18554.
- [27] Omneya Attallah, Skin-CAD: Explainable deep learning classification of skin cancer from dermoscopic images by feature selection of dual high-level CNNs features and transfer learning, *Comput. Biol. Med.* 178 (2024) 108798.
- [28] Zipiao Zhu, Yang Liu, Chang-An Yuan, Xiao Qin, Feng Yang, A diffusion model multi-scale feature fusion network for imbalanced medical image classification research, *Comput. Methods Programs Biomed.* 256 (2024) 108384.
- [29] Ling Tan, Hui Wu, Jingming Xia, Ying Liang, Jining Zhu, Skin lesion recognition via global-local attention and dual-branch input network, *Eng. Appl. Artif. Intell.* 127 (2024) 107385.
- [30] Youyi Song, Yuanlin Liu, Zhizhe Lin, Jinglin Zhou, Duo Li, Teng Zhou, Man-Fai Leung, Learning from AI-generated annotations for medical image segmentation, *IEEE Trans. Consum. Electron.* (2024).
- [31] Jian Li, Jiawei Wang, Fengwu Lin, Wenqi Wu, Zhao-Min Chen, Ali Asghar Heidari, Huiling Chen, DSEUNet: A lightweight unet for dynamic space grouping enhancement for skin lesion segmentation, *Expert Syst. Appl.* (2024) 124544.
- [32] Bill Cassidy, Connah Kendrick, Andrzej Brodzicki, Joanna Jaworek-Korjakowska, Moi Hoon Yap, Analysis of the ISIC image datasets: Usage, benchmarks and recommendations, *Med. Image Anal.* 75 (2022) 102305.
- [33] San-Li Yi, Xue-Lian Yang, Tian-Wei Wang, Fu-Rong She, Xin Xiong, Jian-Feng He, Diabetic retinopathy diagnosis based on RA-EfficientNet, *Appl. Sci.* 11 (22) (2021) 11035.